Les diagrammes fonctionnels

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. Le diagramme de cas d'utilisation 3

III. Exercice : Appliquez la notion 9

IV. Le diagramme de packages 10

V. Exercice : Appliquez la notion 12

VI. Le diagramme d'activité 12

VII. Exercice : Appliquez la notion 19

VIII. Le diagramme de séquence 19

IX. Exercice : Appliquez la notion 26

X. Essentiel 26

XI. Auto-évaluation 26

A. Exercice final 26

B. Exercice : Défi 28

Solutions des exercices 29

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : Local

Pré-requis : Avoir été introduit à l'UML

Contexte

Il existe deux types de diagramme : les diagrammes de structure, utilisés dans les spécifications techniques, et les diagrammes de comportement, utilisés dans les spécifications fonctionnelles. La frontière est parfois mince entre les deux, puisque certains diagrammes de comportement peuvent également être utilisés pour définir une structure. C'est pourquoi il est important de ne pas se limiter à l'un ou à l'autre.

Dans ce cours, nous allons découvrir les principaux diagrammes de comportement : le diagramme de package, le diagramme de cas d'utilisation, le diagramme d'activité et le diagramme de séquence. Nous allons également voir ce que ces diagrammes peuvent apporter aux spécifications fonctionnelles.

II. Le diagramme de cas d'utilisation

Objectifs

- Découvrir le diagramme de cas d'utilisation
- Apprendre toutes ses composantes

Mise en situation

Le premier diagramme que nous allons voir est le **diagramme de cas d'utilisation**. C'est le diagramme le plus utilisé lors de l'élaboration de spécifications fonctionnelles, puisqu'il permet de dresser une liste exhaustive et claire des fonctionnalités de l'application.

Mais plus qu'une simple liste, les diagrammes de cas d'utilisation les plus complets vont également permettre de définir des dépendances entre les différentes fonctionnalités et de gérer leurs accès selon les différents utilisateurs.

Définition **Diagramme de cas d'utilisation**

Un diagramme de cas d'utilisation est un diagramme permettant de définir les fonctionnalités d'une application, leurs pré-requis et les droits d'accès qui y sont liés.

Ce diagramme permet d'avoir une vision précise du périmètre du projet, c'est-à-dire de ce qui va devoir être développé. À partir d'un diagramme de cas d'utilisation, il va être possible d'estimer le temps nécessaire à la réalisation de chaque fonctionnalité et ainsi d'en extrapoler le coût et la date de livraison du projet. Ce diagramme doit donc être exhaustif et validé par le client afin d'éviter tout litige pendant le développement : toute fonctionnalité qui ne fait pas partie du diagramme de cas d'utilisation devra provoquer un nouveau calcul de la facturation et de la date de fin.

Le diagramme de cas d'utilisation est donc bien plus qu'un *listing* de fonctionnalités : c'est un accord entre le client et l'équipe de développement sur le travail à rendre.

Un diagramme de cas d'utilisation est composé de trois éléments principaux : des acteurs, des cas d'utilisation et les relations entre eux.

Méthode **Les acteurs**

Un **acteur** représente une entité externe à l'application qui va être amenée à interagir avec elle. Les principaux acteurs d'une application sont les utilisateurs, mais il existe d'autres types d'acteurs.

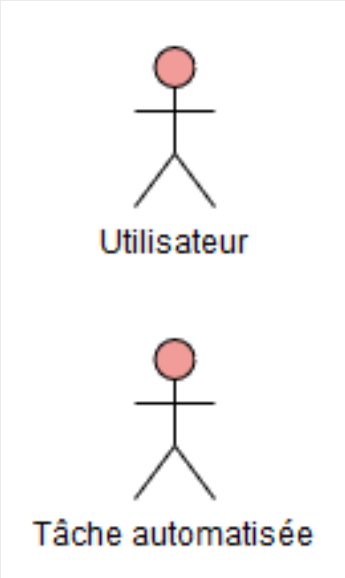
Par exemple, si une fonctionnalité de notre application doit être appelée par une tâche automatisée (comme un envoi de newsletter ou un nettoyage de fichiers), alors l'acteur de cette fonctionnalité sera le système permettant d'automatiser ces tâches. Dans d'autres cas, si notre application est amenée à être appelée par d'autres programmes, par exemple via une API, ceux-ci seront représentés par des acteurs.

Un acteur est souvent lié à une fonctionnalité de connexion, qui peut différer d'un acteur à l'autre : un formulaire pour les utilisateurs, un *token* de connexion pour les API, etc. Identifier les différents acteurs permet donc de définir les différents accès à mettre en place.

Les acteurs sont représentés par un bonhomme bâton et sont placés sur le côté du diagramme.

Exemple

Si notre application est destinée à être utilisée par des utilisateurs, mais que certaines fonctionnalités vont être appelées par une tâche automatisée, alors notre diagramme de cas d'utilisation possédera deux acteurs, symbolisés comme ceci :

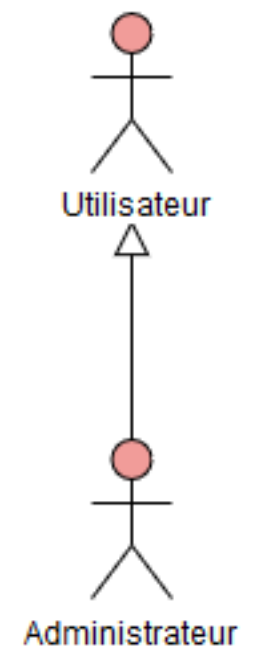


Tous droits réservés à STUDI - Reproduction interdite

Syntaxe

Si un acteur a accès aux mêmes fonctionnalités de base qu'un autre, alors il est possible de définir une **relation de généralisation** entre les acteurs, symbolisée par une flèche à pointe triangulaire fermée.

Ainsi, dans une application où un administrateur possède les mêmes fonctionnalités de base qu'un utilisateur normal, mais a également accès à des fonctionnalités qui lui sont propres, alors il est possible d'établir la relation suivante :



Méthode **Les cas d'utilisation**

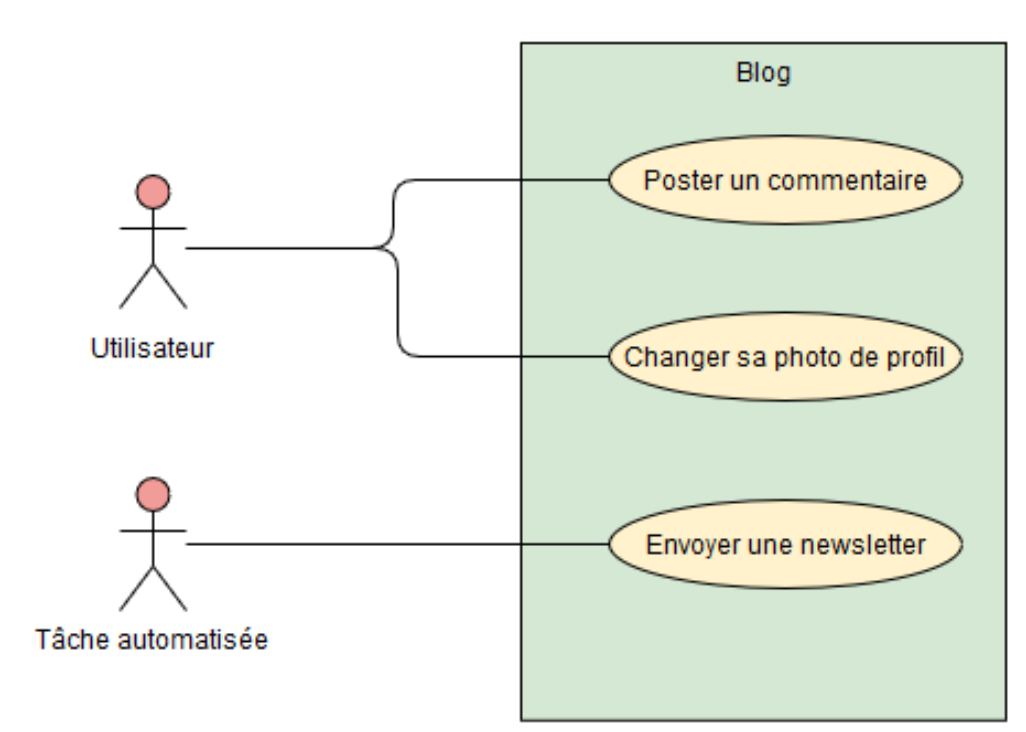
Un **cas d'utilisation** représente une fonctionnalité de l'application qui va être liée à un acteur. Chaque cas d'utilisation est représenté par une ellipse contenant une courte description de la fonctionnalité.

Les cas d'utilisation sont contenus dans un système dont les frontières sont représentées par un rectangle. Si notre application possède des parties indépendantes les unes des autres, alors il existera plusieurs systèmes différents.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Prenons une application de blog où les utilisateurs peuvent poster un commentaire et changer leur photo de profil. Toutes les semaines, une tâche automatisée va également envoyer une newsletter à tous les utilisateurs. Ce système pourra être représenté comme ceci :



Les relations entre les fonctionnalités

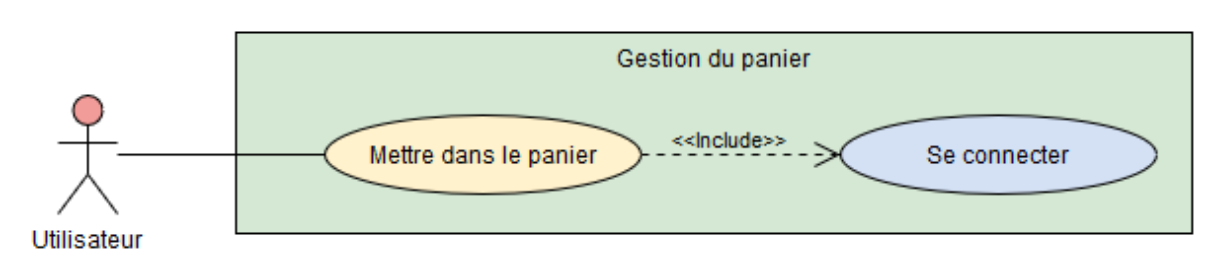
En plus de pouvoir relier des acteurs à des fonctionnalités, il est également possible de relier deux fonctionnalités entre elles. Il existe trois types de relations principales : les inclusions, les extensions et les généralisations.

Méthode **Les inclusions**

Les **inclusions** permettent de définir une dépendance forte : la fonctionnalité incluse doit être réalisée en amont pour que l'autre puisse être appelée. Cette relation est symbolisée par une flèche pointillée portant le label `<<include>>`.

Exemple

Sur un site de vente en ligne, la fonctionnalité « mettre dans le panier » doit inclure « se connecter », si l'on souhaite que l'utilisateur soit identifié au moment où il fait ses achats.



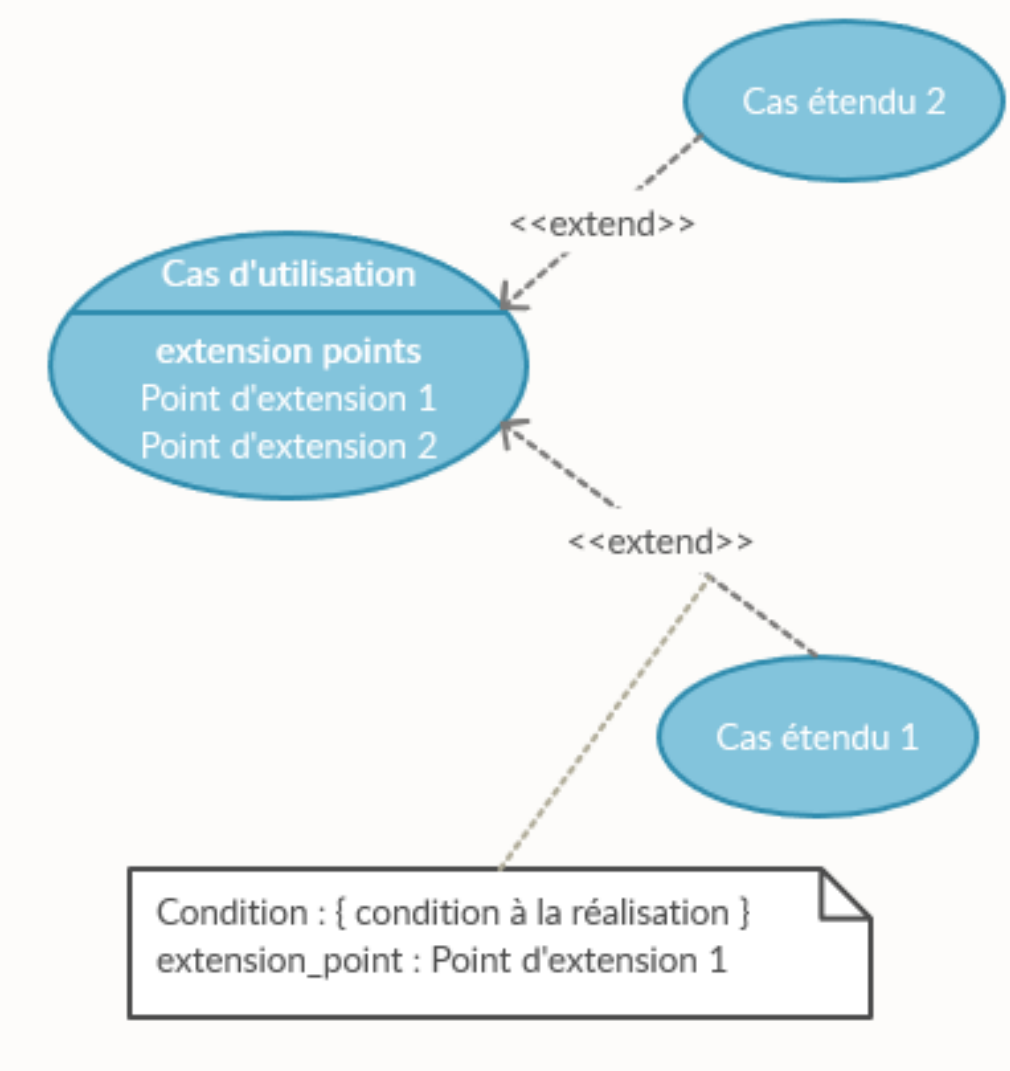
Tous droits réservés à STUDI - Reproduction interdite

Méthode **Les extensions**

Les **extensions**, au contraire des inclusions, permettent de définir une dépendance faible. Si une fonctionnalité en étend une autre, cela signifie qu'elle pourra potentiellement être appelée lorsque l'autre sera exécutée.

Cette relation est symbolisée par une flèche pointillée portant le label `<<extend>>` et par la création d'un point d'extension dans le cas d'utilisation, qui permet d'attribuer un nom à chaque extension. La liste des points d'extension doit être précédée par le titre « `extension points` ».

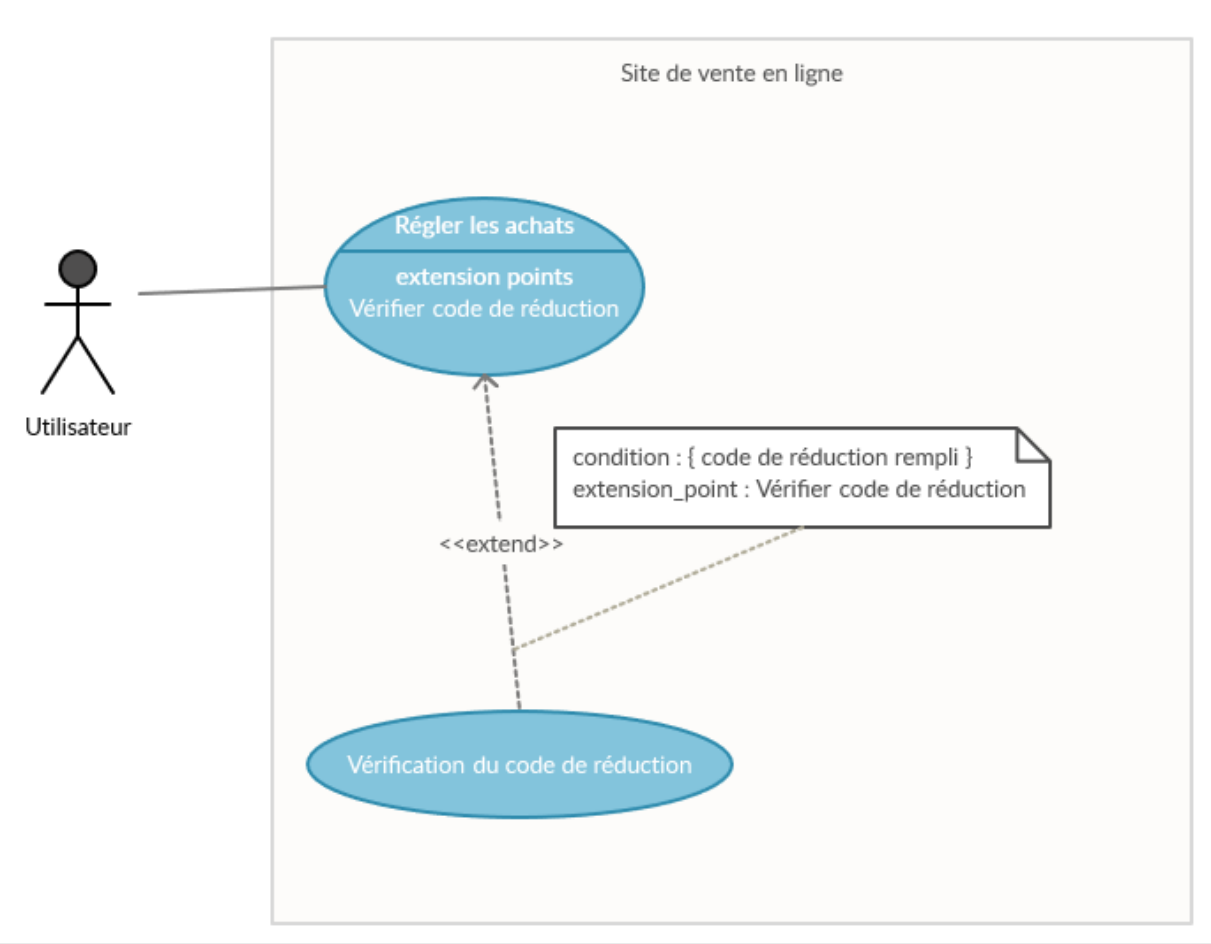
Chaque relation d'extension peut être accompagnée d'une note indiquant à quel point d'extension elle fait référence, et quelle est la condition d'exécution du cas d'utilisation étendu.



Tous droits réservés à STUDI - Reproduction interdite

Exemple

Au moment où l'utilisateur va devoir payer sa commande sur le site de vente en ligne, il a la possibilité de rentrer un coupon de réduction. S'il a été rempli, il va falloir le vérifier et appliquer la réduction nécessaire.



Remarque

Il est possible d'utiliser une relation d'extension sans pour autant avoir de conditions ni de points d'extension. Ils sont parfois utiles pour préciser la relation, mais peuvent être omis si la relation de base est assez claire.

Complément

Les labels `<<include>>` et `<<extend>>` sont appelés des « stéréotypes ». Leur rôle est d'expliciter la sémantique de la flèche pointillée, qui symbolise une dépendance entre deux fonctionnalités. Ce sont les deux stéréotypes les plus utilisés, mais il en existe d'autres, qui apparaissent dans d'autres diagrammes.

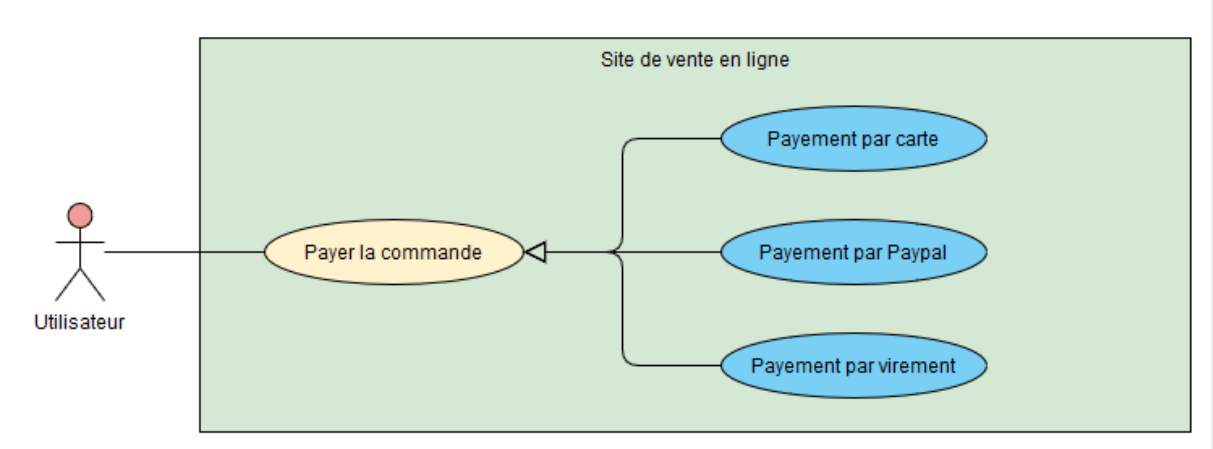
Méthode **La généralisation**

La **généralisation** permet de préciser qu'un cas d'utilisation est une variante d'un autre. Le cas général peut ainsi être substitué à sa spécialisation. La généralisation peut s'apparenter au principe d'héritage des langages de programmation orientés objet et permet de créer des cas d'utilisation généraux qui pourront être précisés. La généralisation est symbolisée par une flèche pleine à la pointe blanche.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Sur notre site de vente en ligne, l'utilisateur peut choisir entre plusieurs modes de paiement : par carte, par virement ou par Paypal. Il est donc possible d'avoir un cas d'utilisation général « Payer la commande » et un cas spécifique par moyen de paiement :



Syntaxe **À retenir**

- Le diagramme de cas d'utilisation est la base des spécifications fonctionnelles. Il permet d'établir une liste de fonctionnalités et des acteurs qui y ont accès.
- Il est également possible de lier les cas d'utilisation entre eux avec trois relations différentes : l'inclusion, l'extension et la généralisation.

Complément

Les diagrammes de cas d'utilisation[1]

Des exemples de diagrammes de cas d'utilisation[2]

III. Exercice : Appliquez la notion

Question

[solution n°1 p.31]

Un cabinet médical a besoin d'une application permettant de gérer le suivi de ses patients. Il sera utilisé aussi bien par les secrétaires que par les médecins, mais les deux comptes n'auront pas les mêmes fonctionnalités.

Les secrétaires vont pouvoir ajouter des rendez-vous dans le planning, mais, pour chaque ajout, il faudra vérifier en amont que le créneau soit disponible.

Les médecins, quant à eux, ne se chargent pas du planning, mais doivent pouvoir créer une ordonnance pour un patient. Lors de la création d'une ordonnance, ils peuvent choisir de l'envoyer directement à la pharmacie du patient, par e-mail ou par fax.

Enfin, les médecins et les secrétaires doivent pouvoir créer la fiche d'un patient dans l'application.

Réalisez le diagramme de cas d'utilisation de ce projet.

Indice :

On peut distinguer deux acteurs : les secrétaires et les médecins.

1 https://www.uml-diagrams.org/use-case-diagrams.html
2 https://circle.visual-paradigm.com/category/uml-diagrams/use-case/

Tous droits réservés à STUDI - Reproduction interdite

Indice :

Le cas d'utilisation « ajouter un rendez-vous » inclut une vérification de la disponibilité.

Indice :

La création d'une ordonnance est étendue par son envoi à la pharmacie.

Indice :

L'envoi d'une ordonnance est un cas d'utilisation général qui possède deux spécifications : l'envoi par e-mail et l'envoi par fax.

IV. Le diagramme de packages

Objectifs

- Découvrir le diagramme de packages
- Savoir dans quelles situations il faut l'utiliser

Mise en situation

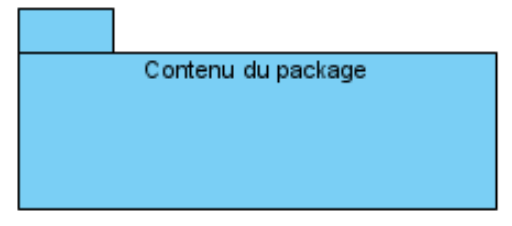
Le diagramme de cas d'utilisation est très pratique pour détailler chaque fonctionnalité et leur lien par rapport aux autres. Cependant, pour un projet possédant un périmètre très important, la grande quantité de diagrammes nécessaires pour tout décrire peut rendre les spécifications confuses et difficiles à lire. Heureusement, il existe un diagramme permettant de prendre du recul par rapport au diagramme de cas d'utilisation afin d'avoir une vision d'ensemble du projet : le diagramme de paquetages.

Définition **Diagramme de paquetages**

Le **diagramme de paquetages** (aussi appelé diagramme de packages) permet d'organiser des éléments, quels qu'ils soient, en les regroupant dans des ensembles, appelés « paquetages ». Il peut ainsi être utilisé dans de nombreux cas, mais on peut en distinguer deux principaux : regrouper des cas d'utilisation en différentes parties afin d'avoir une vue d'ensemble du projet, et structurer les différents classes issues d'un diagramme de classes. Ce diagramme a donc sa place à la fois dans des spécifications fonctionnelles et techniques.

Méthode

Les paquetages sont représentés par un grand rectangle surplombé d'un rectangle plus petit. Le contenu du package est indiqué dans le rectangle principal. Selon l'utilisation du diagramme, le contenu pourra prendre des formes différentes : une liste de classes pour les spécifications techniques ou une description pour les spécifications fonctionnelles.



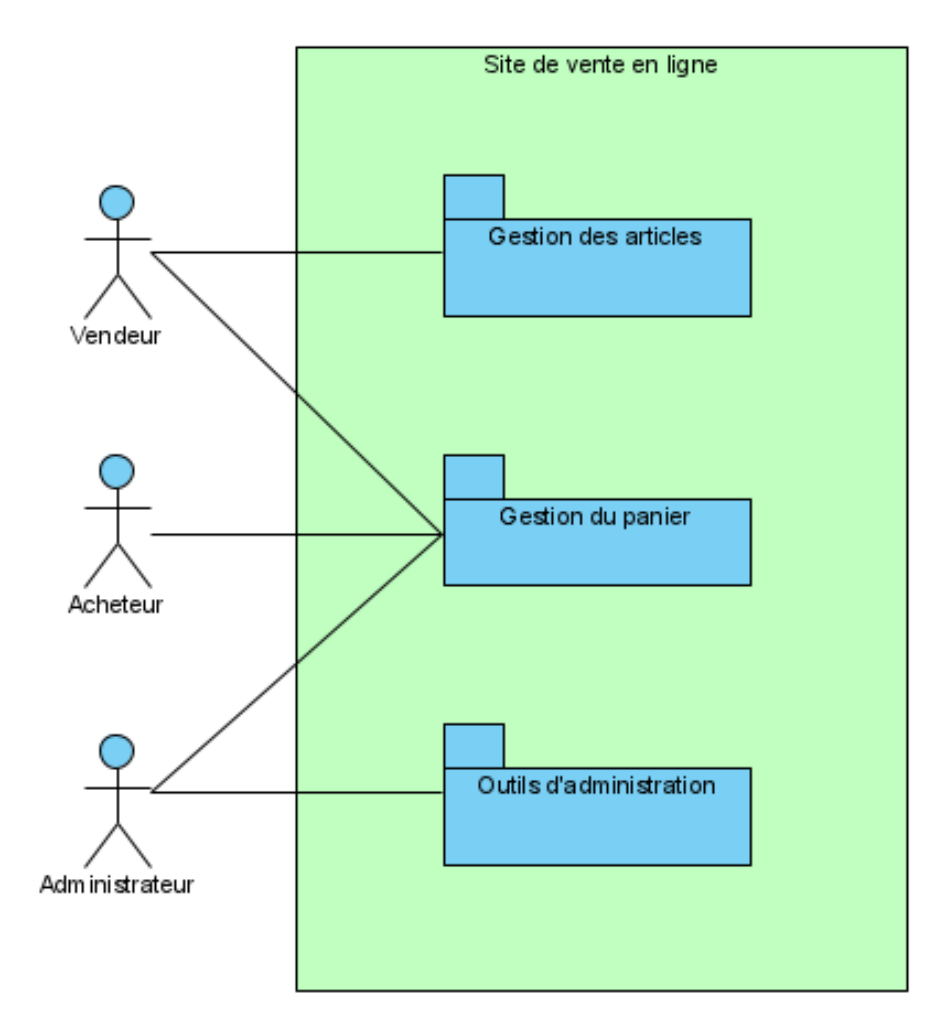
Tout comme les diagrammes de cas d'utilisation, les packages sont contenus dans un système et peuvent être reliés à des acteurs.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Un site de vente en ligne possède plusieurs ensembles de fonctionnalités : la gestion du panier de l'acheteur (ajouter/supprimer des articles dans son panier, vider tout le panier, sauvegarder le panier...), la gestion des articles du vendeur (ajouter/supprimer un article, modifier la description, les photos...) et les outils d'administration (visualiser le trafic du site, les statistiques, la charge du serveur...). Le vendeur et l'administrateur ont accès à toutes les fonctionnalités d'achat.

Ces trois ensembles peuvent être illustrés par le diagramme suivant :



Syntaxe **À retenir**

- Le diagramme de paquetages permet de regrouper des éléments dans des packages. Il peut être utilisé pour toutes sortes de choses, les deux cas les plus communs étant les regroupements des cas d'utilisation et ceux des classes.

Complément

Les diagrammes de paquetages[1]

1 https://www.uml-diagrams.org/package-diagrams-overview.html

Tous droits réservés à STUDI - Reproduction interdite

V. Exercice : Appliquez la notion

Question

[solution n°2 p.31]

Une salle de sport souhaiterait avoir un logiciel permettant de gérer son activité. La salle propose un abonnement annuel à ses clients, qu'ils peuvent résilier à tout moment. Le point fort de la salle est que les clients sont encadrés par des coachs qui sont à leur disposition lors de « rendez-vous coaching ». Le logiciel sera utilisé par les coachs sportifs de la salle, mais également par les clients. Voici la liste des fonctionnalités souhaitées :

- Un client doit pouvoir se créer un profil pour renseigner ses informations (nom, prénom, âge, poids, informations médicales...).
- Un client doit pouvoir demander un « rendez-vous coaching » avec un coach.
- Un client doit pouvoir s'abonner (ou renouveler son abonnement) depuis l'application.
- Un coach et un client doivent pouvoir annuler un rendez-vous coaching.
- Un client doit pouvoir résilier son abonnement depuis l'application.
- Un client doit pouvoir modifier les informations de son profil.
- Un coach doit pouvoir accéder à son planning du jour (donc ses « rendez-vous coaching »).

Regroupez ces fonctionnalités en paquetages et créez le diagramme correspondant.

Indice :

On peut regrouper toutes ces fonctionnalités en 3 paquetages

VI. Le diagramme d'activité

Objectifs

- Découvrir le diagramme d'activité
- Connaître ses différents composants
- Comprendre son utilité dans les spécifications fonctionnelles

Mise en situation

Comme nous l'avons vu, un cas d'utilisation est un élément permettant de représenter une fonctionnalité dans son ensemble. Cependant, dans la pratique, une fonctionnalité est souvent composée de plusieurs étapes successives, comme « se rendre sur une page », « remplir un champ du formulaire » ou encore « appuyer sur le bouton de validation ». Pour modéliser ces étapes, on utilise alors le **diagramme d'activité**.

Définition **Diagramme d'activité**

Le diagramme d'activité est un diagramme permettant une décomposition en étapes d'un cas d'utilisation. Il est utilisé pour décrire le processus de résolution d'une fonctionnalité complexe afin que le client et l'équipe de développement puissent visualiser précisément son déroulement.

Un diagramme d'activité est composé de différents nœuds, chacun représentant une étape, liés entre eux par des transitions.

Tous droits réservés à STUDI - Reproduction interdite

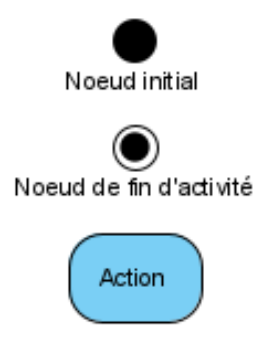
Méthode **Les nœuds de base**

Un diagramme d'activité est composé d'au moins trois nœuds de base.

Le premier est le **nœud initial**, qui représente le point de départ de la fonctionnalité. Il est modélisé par un petit cercle noir.

Le second est le **nœud de fin d'activité**, qui symbolise le point d'arrivée du cas d'utilisation. Il est identique au nœud initial, mais possède un contour noir supplémentaire.

Enfin, chaque étape est représentée par un **nœud d'action**, sous la forme d'une ellipse contenant une courte description de l'étape.



Pour se déplacer d'un nœud à un autre, il faut utiliser des transitions, représentées par des flèches. Les transitions servent à définir l'ordre des différentes actions. Un nœud initial ne peut avoir qu'une transition sortante, un nœud de fin d'activité ne peut pas avoir de transition sortante, et une action doit avoir une seule transition entrante et une seule transition sortante.

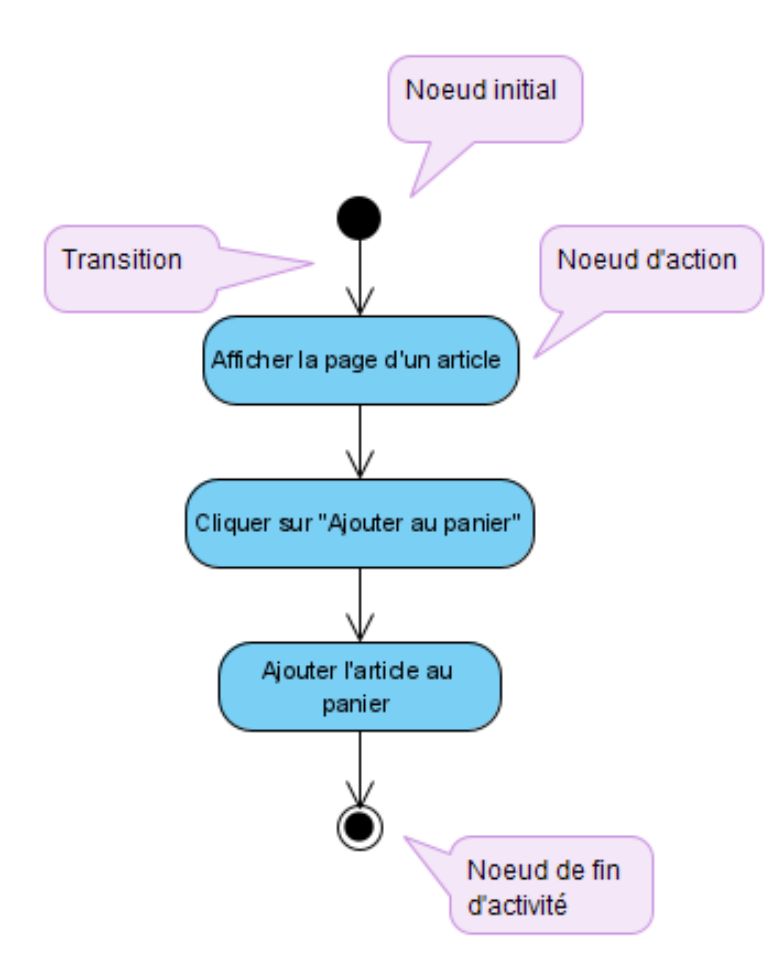
Remarque

Si cela rend le diagramme plus clair, il est possible de mettre plusieurs nœuds de fin d'activité.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

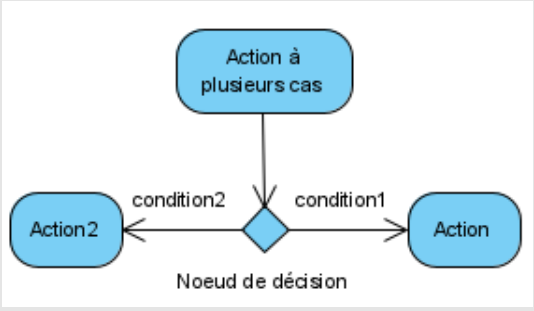
Sur un site de vente en ligne, un utilisateur peut ajouter des articles dans son panier. Ce cas d'utilisation peut être découpé en plusieurs étapes : l'affichage de la page de l'article, le clic de l'utilisateur sur le bouton « Ajouter au panier » et l'ajout de l'article dans le panier. Voici l'illustration de ce découpage :



Méthode **Le nœud de décision**

Certains cas d'utilisation peuvent avoir plusieurs scénarios possibles pendant leur exécution. Reprenons l'exemple du site de vente en ligne ci-dessus, en lui rajoutant une étape : l'article que l'on met dans le panier doit être en stock. Il va donc falloir vérifier si l'article est disponible, ce qui va diviser notre diagramme en deux cas : s'il est disponible, alors on l'ajoute au panier, sinon, on affiche un message d'erreur.

Pour illustrer cette séparation dans un diagramme d'activité, il faut y ajouter un **nœud de décision**, sous la forme d'un losange. Plusieurs transitions peuvent partir d'un nœud de décision : il y aura donc une flèche par cas possible. Chaque flèche doit être accompagnée d'un texte indiquant dans quelle condition utiliser la transition.

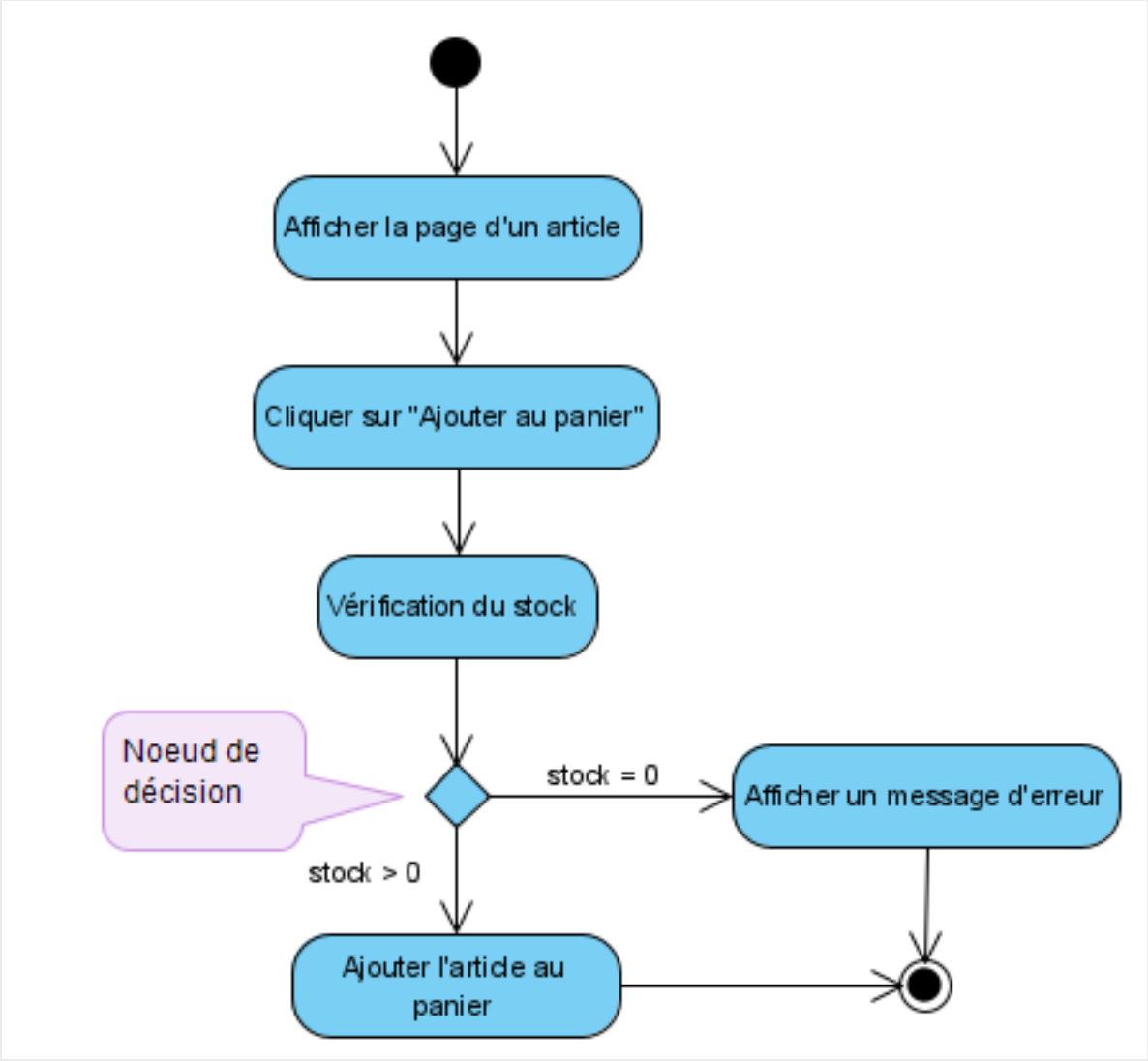


Tous droits réservés à STUDI - Reproduction interdite

On peut voir un nœud de décision comme un `switch`, chaque transition sortante étant un `case`.

Exemple

Rajoutons la vérification des stocks dans notre site de vente :



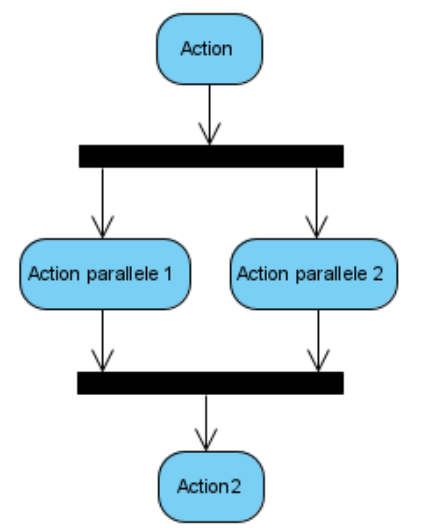
Méthode **Bifurcation et fusion**

Il peut arriver que certaines actions du diagramme puissent se faire en parallèle, de sorte que l'ordre de leur exécution n'importe pas. Il est alors possible de séparer notre diagramme en deux branches grâce à un **nœud de bifurcation**, puis de regrouper ces deux branches plus loin grâce à un **nœud de fusion**.

Tous droits réservés à STUDI - Reproduction interdite

Ces nœuds sont symbolisés par un épais trait noir. La différence entre les deux est qu'un nœud de bifurcation ne possède qu'une transition entrante et plusieurs transitions sortantes, tandis qu'un nœud de fusion possède plusieurs transitions entrantes et une seule sortante.

Le nœud de fusion permet d'attendre que toutes les actions en amont soient terminées avant de continuer. Sur le schéma de droite, **Action2** ne sera exécutée que lorsque les deux actions en parallèle auront fini leur tâche.



Le nœud de bifurcation se différencie du nœud de décision par le fait que, lors d'une bifurcation, toutes les actions seront exécutées, tandis que le nœud de décision ne choisit qu'une seule action selon certaines conditions.

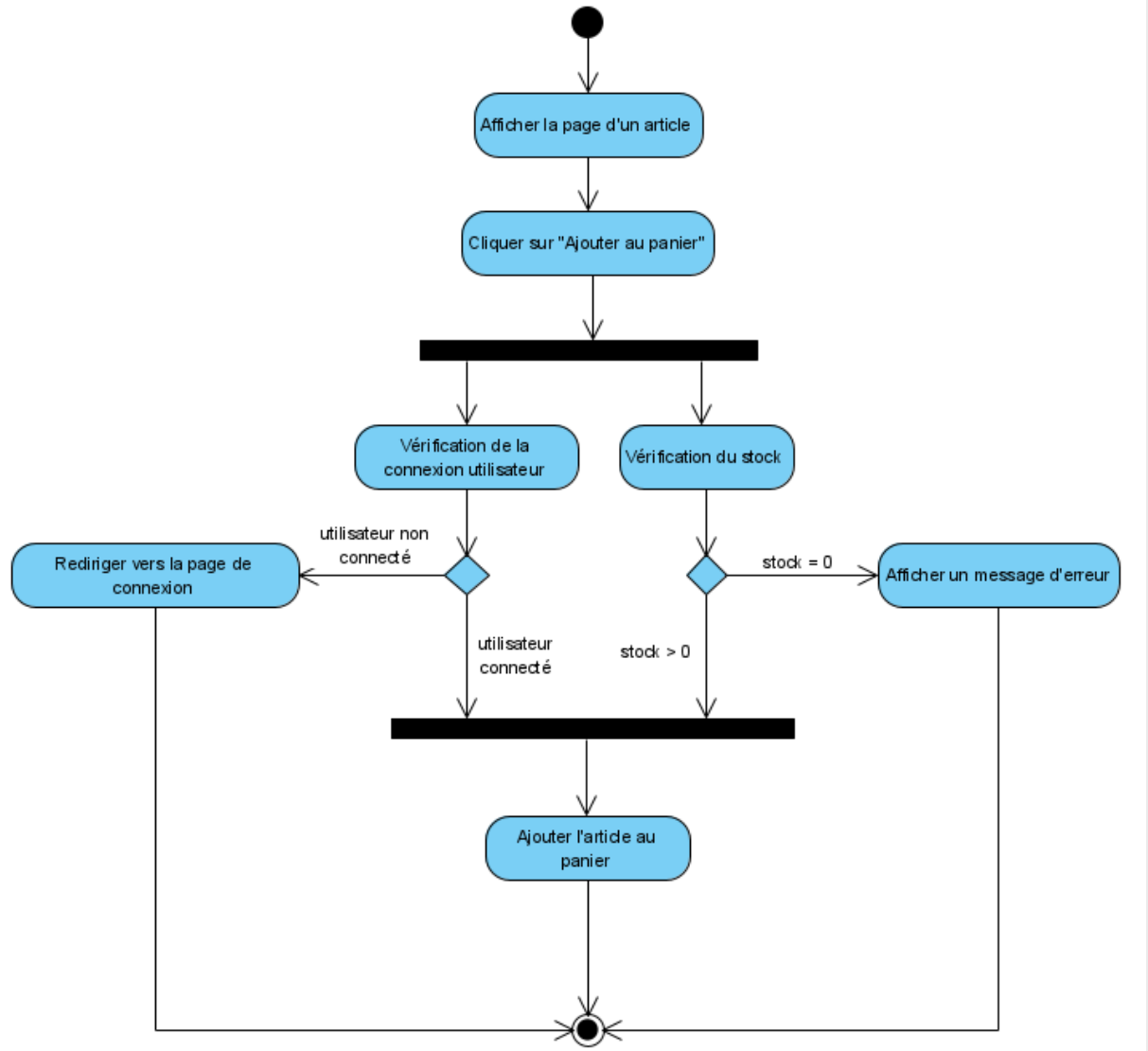
Remarque

Dans le code final, les actions issues d'une bifurcation ne seront pas forcement exécutées en parallèle : cela dépend des technologies utilisées et de l'environnement de développement du projet. La sémantique première d'une bifurcation est que l'ordre de ces actions n'importe pas : elles peuvent être exécutées dans n'importe quel ordre, en parallèle ou non.

Exemple

En plus de vérifier si l'article est en stock, nous devons aussi nous assurer que l'utilisateur est actuellement connecté. Ces deux vérifications ne doivent pas forcément être réalisées dans un ordre précis : nous pouvons donc utiliser la bifurcation pour réaliser ces traitements en parallèle. Cependant, il va falloir attendre que ces deux vérifications soient effectuées pour pouvoir ajouter l'article dans le panier : il faudra donc les fusionner en amont de cette action.

Tous droits réservés à STUDI - Reproduction interdite



Complément **Une variante : le diagramme états-transitions**

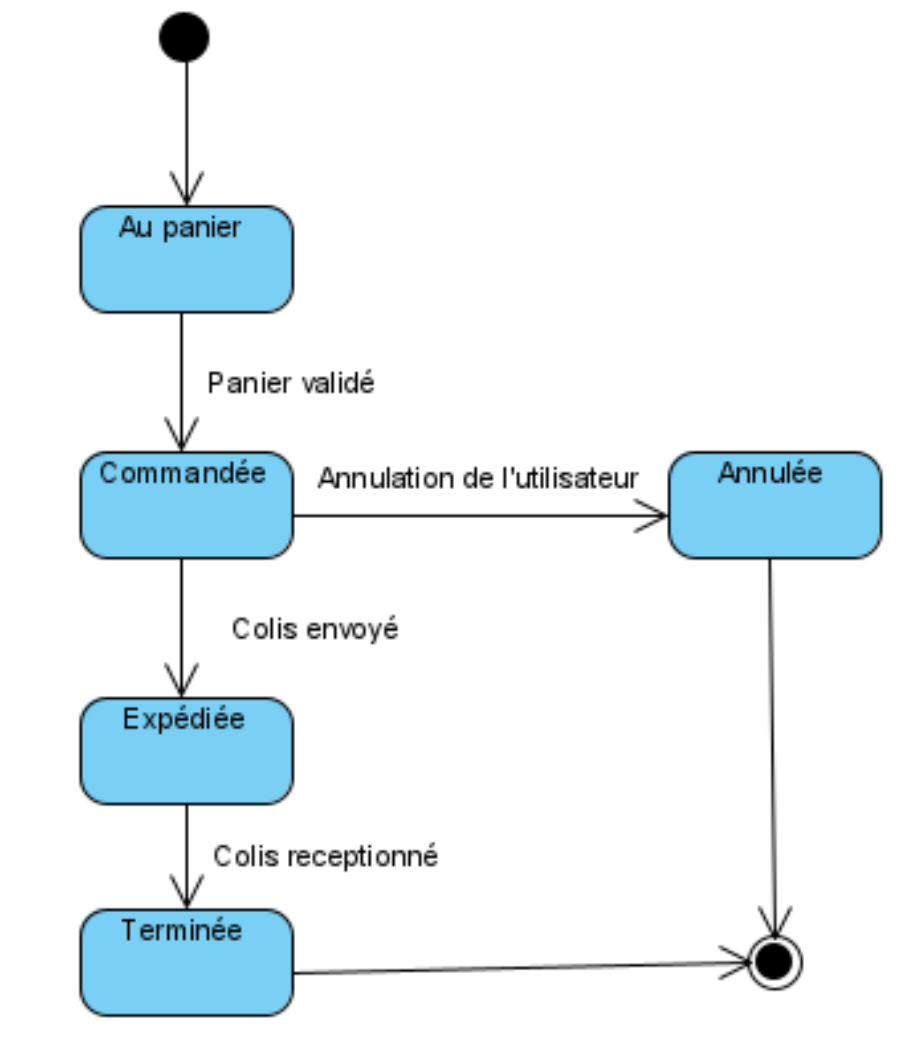
Il arrive que certains éléments d'une application se basent sur des états internes, qui vont décider des fonctionnalités qui vont pouvoir être exécutées. Par exemple, on peut découper la vie d'une commande d'un site de vente en ligne en plusieurs états : « Au panier », « Commandée », « Expédiée », « Annulée » et « Terminée ». Tous les cas d'utilisation ne seront pas accessibles à tous les états, et c'est l'exécution de certaines fonctionnalités qui fera passer la commande d'un état à l'autre. Cet enchaînement d'états peut parfois être complexe, mais il existe un diagramme UML permettant de clarifier ces interactions : le **diagramme états-transitions**.

Le diagramme états-transitions peut être vu comme une variante du diagramme d'activité : on y retrouve un état initial, un état final, des états et des transitions qui utilisent les mêmes symboles. Cependant, ces deux diagrammes ont des utilisations complètement différentes : plutôt que de se focaliser sur un cas d'utilisation, le diagramme états-transitions va se concentrer sur une entité de l'application et décrire comment certaines fonctionnalités vont faire la transition entre ces états.

Dans la pratique, cette différence est exprimée par le fait que l'action permettant de passer d'un état à un autre est toujours indiquée sur la flèche représentant la transition. De plus, les nœuds de décision ne sont pas nécessaires : un état peut avoir plusieurs transitions, liées à des actions différentes.

Tous droits réservés à STUDI - Reproduction interdite

Voici un exemple de diagramme états-transitions représentant la vie d'un panier d'un site de vente en ligne :



Syntaxe **À retenir**

- Le diagramme d'activité permet de préciser toutes les étapes permettant de réaliser un cas d'utilisation. Il est composé de trois nœuds de base : le nœud initial, le nœud de fin d'activité, et le nœud d'action. En plus de ces nœuds, il est possible de rajouter des conditions à des étapes grâce au nœud de décision ou de les réaliser en parallèle grâce aux nœuds de bifurcation et de fusion.
- Le diagramme états-transitions est une variante du diagramme d'activité. Il permet de représenter l'évolution des états d'un composant de manière globale, plutôt que de se concentrer sur un cas d'utilisation.

Complément

Les diagrammes d'activité[1]

1 https://www.uml-diagrams.org/activity-diagrams.html

Tous droits réservés à STUDI - Reproduction interdite

VII. Exercice : Appliquez la notion

Question

[solution n°3 p.32]

Un client souhaiterait créer une application de partage d'images. L'un des cas d'utilisation de cette application est « Envoyer une image sur le serveur » : les utilisateurs disposent d'un formulaire permettant de choisir une image de leur ordinateur pour l'envoyer sur le serveur. Seules les images de type jpg sont autorisées et elles ne doivent pas peser plus de 1 Mo : si c'est le cas, alors un message d'erreur s'affiche et l'image n'est pas envoyée.

Réalisez le diagramme d'activité de ce cas d'utilisation.

Indice :

La vérification du poids et celle du type n'ont pas forcément d'ordre d'exécution précis.

VIII. Le diagramme de séquence

Objectifs

- Découvrir le diagramme de séquence
- Connaître ses différents composants
- Comprendre son utilité dans les spécifications fonctionnelles

Mise en situation

Un cas d'utilisation implique diverses interactions entre les acteurs d'un système. Ces interactions sont parfois simples, comme un utilisateur qui demande à une page statique de s'afficher, mais elles peuvent rapidement se complexifier. En effet, notre application devra souvent interroger un système externe, comme une base de données ou une API. De plus, il peut arriver que les actions d'autres acteurs puissent être nécessaires à la réalisation d'un cas d'utilisation, comme la validation par un modérateur d'un commentaire posté par un utilisateur. Modéliser ces interactions et indiquer leur ordre d'exécution peut ainsi permettre de mieux comprendre le fonctionnement d'un système. C'est pourquoi l'UML propose le **diagramme de séquence**.

Définition **Diagramme de séquence**

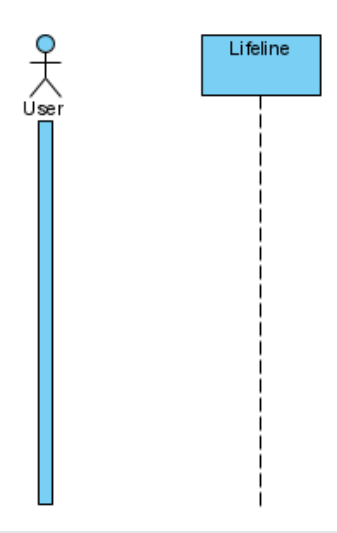
Le diagramme de séquence est un diagramme permettant d'illustrer, pour un cas d'utilisation, toutes les interactions entre les différents acteurs, comme l'utilisateur, l'application elle-même et tous les systèmes externes nécessaires. Il permet d'avoir une vision détaillée de toutes les communications effectuées pendant l'exécution d'une fonctionnalité, dans l'ordre chronologique.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Acteurs et lignes de vie**

Le diagramme de séquence s'écrit sous forme de colonnes, une par acteur, et se lit de haut en bas. Contrairement aux autres diagrammes en UML, seul l'acteur principal (c'est-à-dire celui qui déclenche la fonctionnalité) est symbolisé par un bonhomme bâton : les autres sont écrits sous forme de rectangles contenant leurs noms.

Chaque acteur possède sa propre ligne de vie, qui est une ligne en pointillés permettant d'exprimer les moments où l'acteur est actif. Les périodes d'activité sont représentées par des rectangles pleins. À noter que l'utilisateur principal est toujours actif, puisqu'il commence l'exécution de la fonctionnalité et qu'il attend de recevoir la réponse finale.



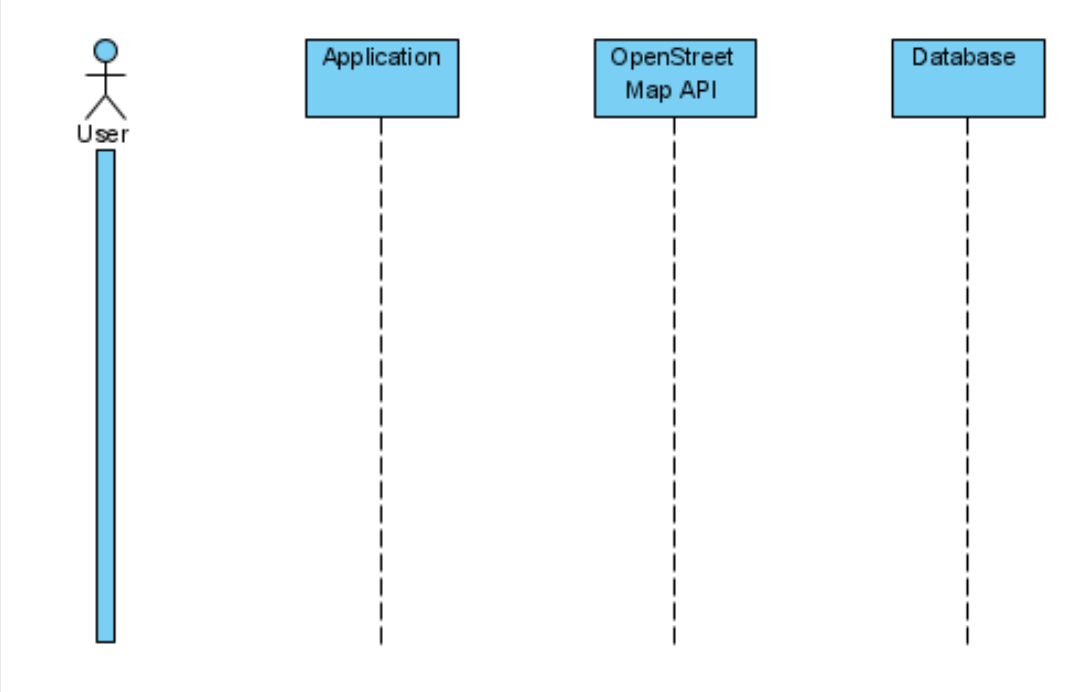
Exemple

Pour illustrer l'utilité d'un diagramme de séquence, prenons l'exemple d'une application de géolocalisation simpliste : l'utilisateur saisit une adresse, et l'application va retourner les coordonnées GPS correspondant à cette adresse. Pour cela, notre application va se baser sur l'API d'OpenStreetMap, qui est un service de cartographie *open-source*.

De plus, pour des raisons de performance, nous allons également stocker les résultats dans notre propre base de données. Ainsi, si la même adresse est demandée plusieurs fois, alors la première requête utilisera l'API d'OpenStreetMap et insérera le résultat en base, et les autres ne feront que retourner le résultat précédemment stocké.

Tous droits réservés à STUDI - Reproduction interdite

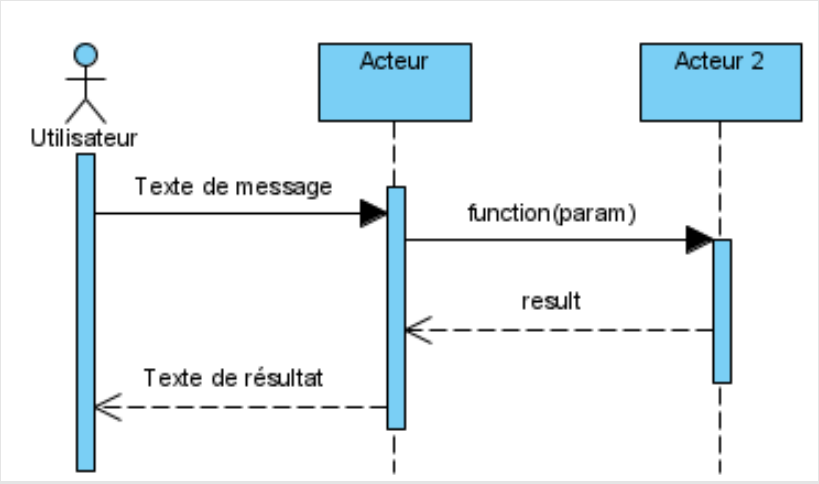
Ce cas d'utilisation met donc en scène 4 acteurs : l'utilisateur, qui est l'acteur principal ; l'application, qui va orchestrer tous les appels ; l'API d'OpenStreetMap ; et notre base de données. La base de notre diagramme va donc être :



Méthode **Messages**

Chaque acteur va communiquer avec les autres grâce à des messages, représentés par des flèches accompagnées d'un texte indiquant la nature du message. Ce texte peut être sous la forme d'une simple description ou indiquer l'appel à une fonction.

Lorsqu'un acteur envoie un message à un autre, il attend généralement une réponse, qui peut soit être une valeur, soit un simple code indiquant si la requête a bien été effectuée. Elle est symbolisée par une flèche pointillée accompagnée d'un texte décrivant la réponse. Les deux acteurs deviennent alors actifs tant que la réponse a été générée, ce qui modifie l'apparence de leur ligne de vie en conséquence : les pointillés sont remplacés par des rectangles pleins.



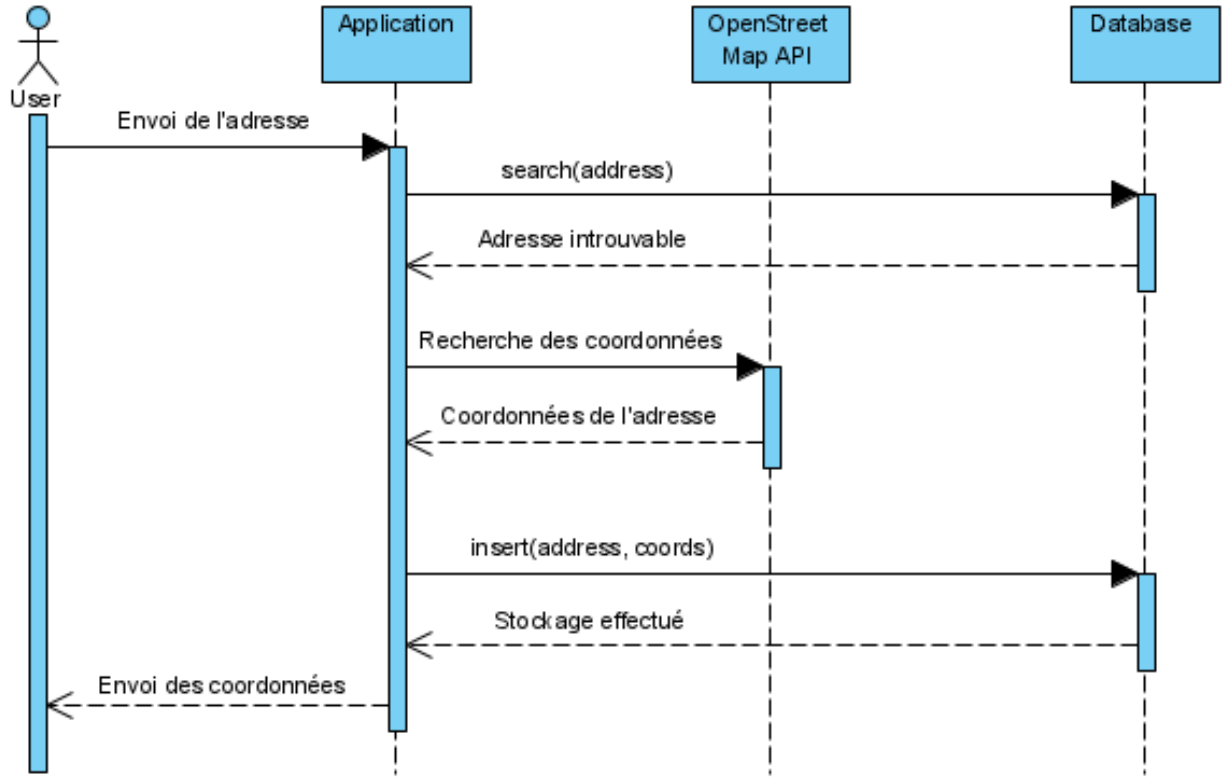
Tous droits réservés à STUDI - Reproduction interdite

Exemple

Listons toutes les étapes de notre application de cartographie dans leur ordre chronologique :

1. L'utilisateur remplit un champ avec une adresse dont il souhaite connaître les coordonnées GPS
2. L'application va vérifier si cette adresse existe en base de données (pour l'exemple, nous prenons le cas où elle n'existe pas)
3. L'application va récupérer les coordonnées sur l'API d'OpenStreetMap
4. L'application va stocker l'adresse et les coordonnées en base de données
5. L'application va retourner les coordonnées à l'utilisateur

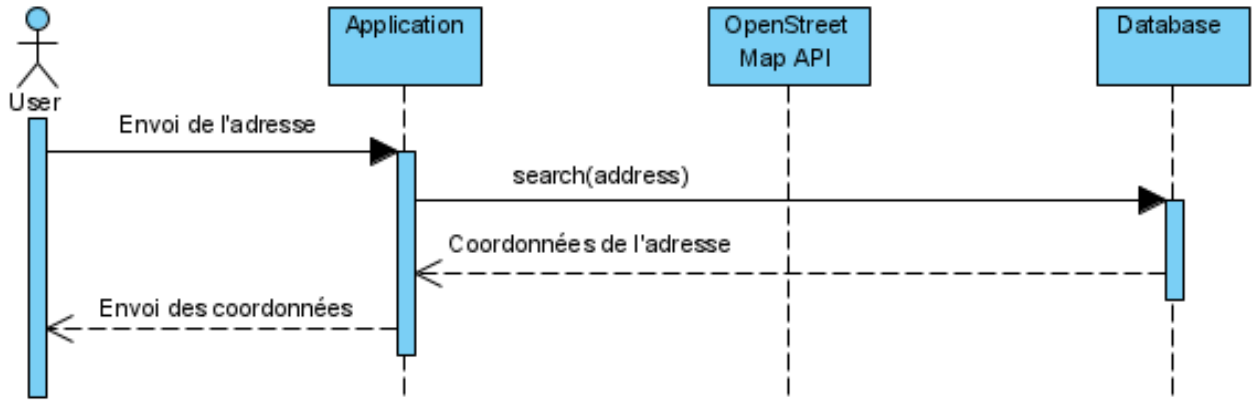
Ce cas donne le diagramme de séquence suivant :



Tous droits réservés à STUDI - Reproduction interdite

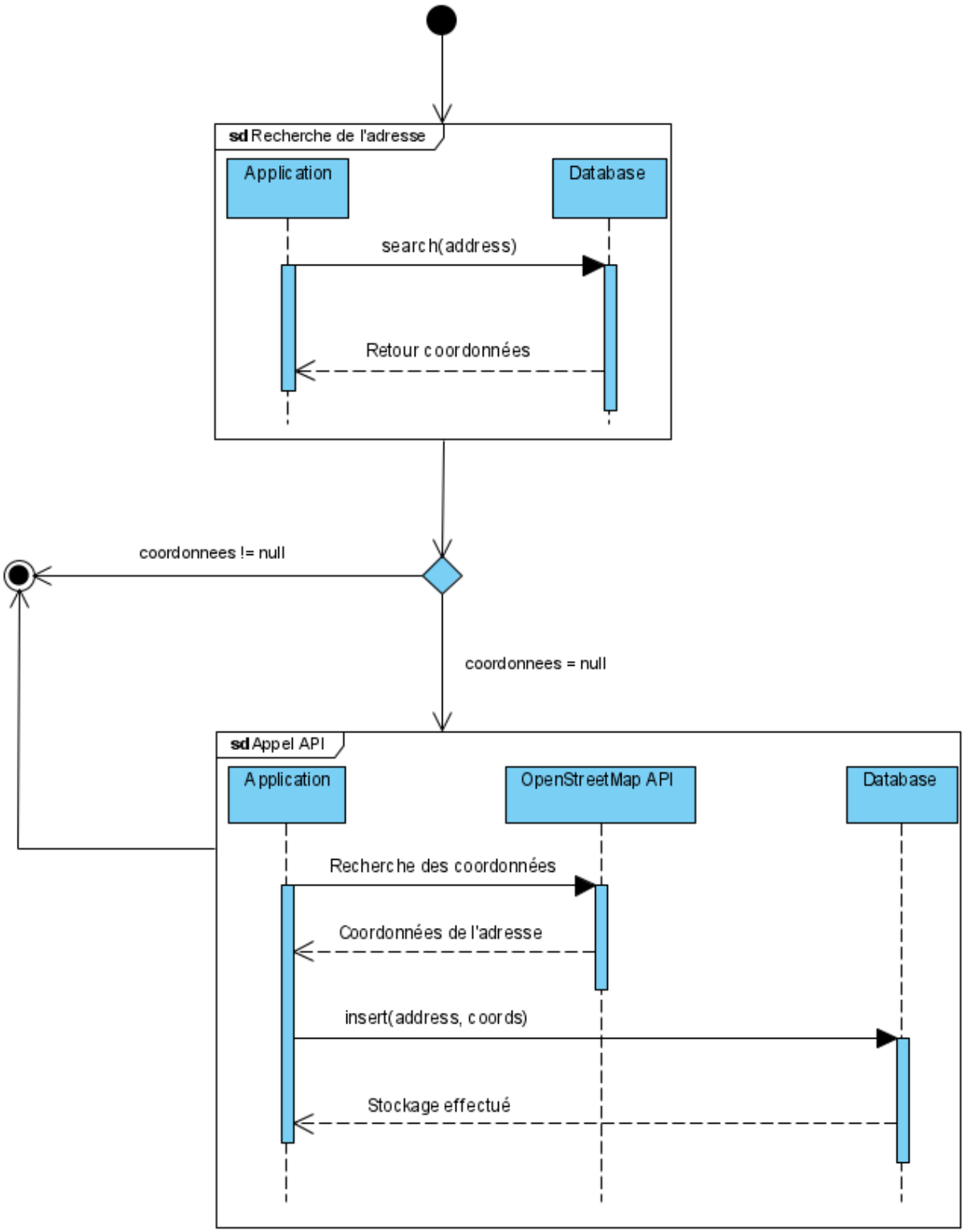
Remarque

Un diagramme de séquence ne représente qu'un seul scénario possible du cas d'utilisation. Dans notre exemple, si l'on souhaite également illustrer le cas où l'adresse est trouvée en base de données, alors il faut réaliser un autre diagramme de séquence :



Tous droits réservés à STUDI - Reproduction interdite

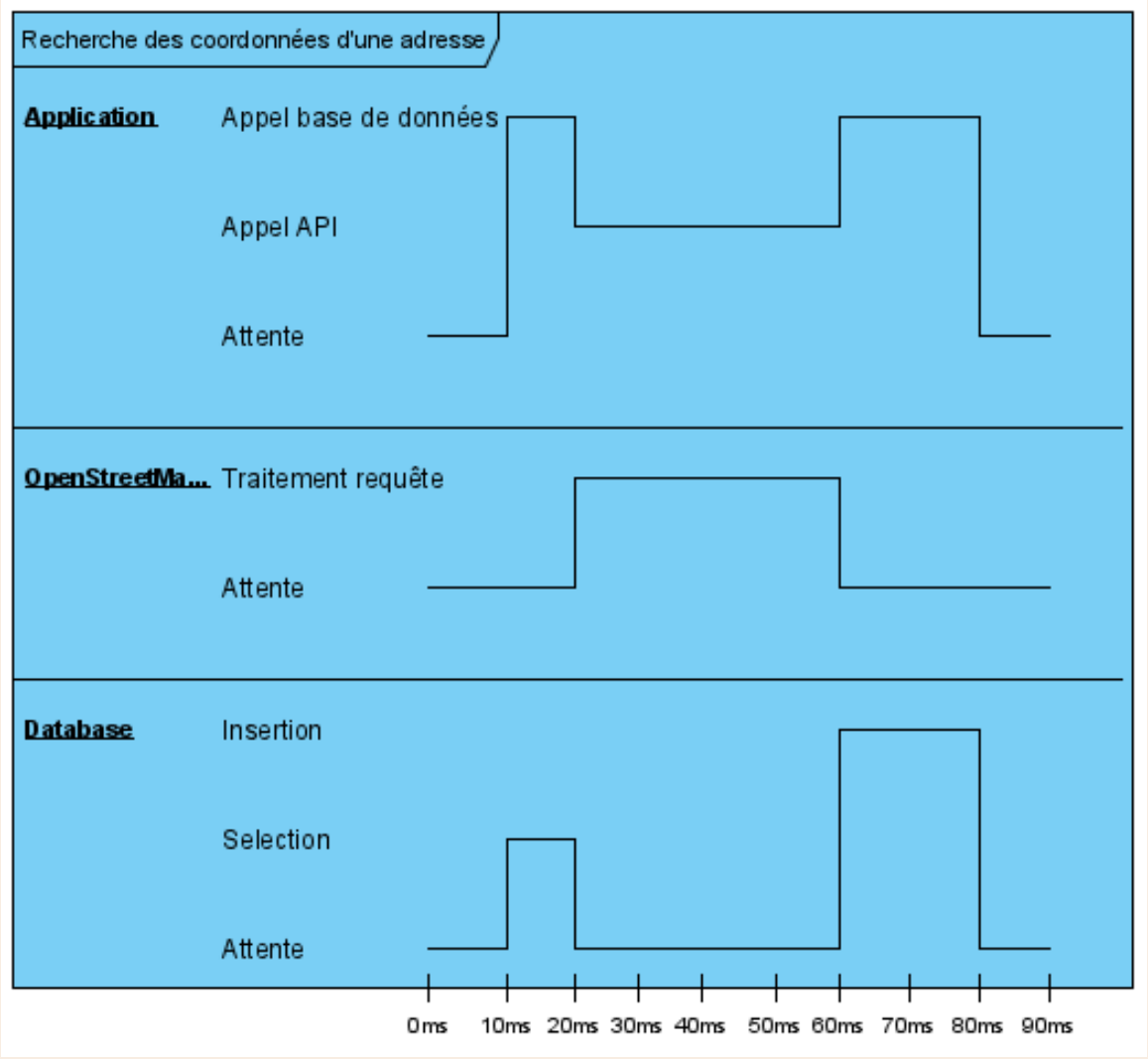
Il est cependant possible de combiner un diagramme d'activité et plusieurs diagrammes de séquence pour faire un **diagramme global d'interactions**. Ce diagramme permet de regrouper tous les scénarios en un seul diagramme en utilisant les éléments de logique d'un diagramme d'activité :



Tous droits réservés à STUDI - Reproduction interdite

Complément **Une variante : le diagramme de temps**

Le **diagramme de temps** est une variante du diagramme de séquence qui se lit horizontalement, de gauche à droite, et qui se concentre sur l'évolution des états de chaque acteur dans le temps. Contrairement au diagramme de séquence, le diagramme de temps utilise un axe temporel précis permettant d'indiquer le temps d'exécution de chaque étape.



Le diagramme de temps permet de mettre en valeur les étapes qui peuvent nuire aux performances de l'application. Dans l'exemple ci-dessus, on peut voir que l'étape la plus longue est le traitement de la requête par OpenStreetMap. On peut imaginer que c'est ce constat qui a mené à la mise en place du système de stockage des adresses en base de données.

Syntaxe **À retenir**

- Le diagramme de séquence permet de représenter les interactions entre les différents acteurs. Ces interactions sont représentées par des messages : un acteur envoie un message à un autre, puis attend la réponse. Ce diagramme permet ainsi de visualiser les communications entre notre application et les autres systèmes, mais aussi de mesurer l'activité de chaque acteur.
- Le diagramme de temps est une variante du diagramme de séquence qui met l'emphase sur le temps de chaque étape de l'exécution du programme.

Tous droits réservés à STUDI - Reproduction interdite

Complément

Les diagrammes de séquence[1]

Les diagrammes de temps[2]

IX. Exercice : Appliquez la notion

Question

[solution n°4 p.33]

Un site de vente en ligne veut permettre à ses utilisateurs de saisir leurs informations bancaires directement sur le site afin de payer leurs articles. Pour cela, le site va devoir faire appel à l'API d'une banque. Voici les étapes à réaliser pour procéder au paiement :

1. L'utilisateur remplit un formulaire avec ses informations bancaires.
2. On appelle l'API de la banque pour vérifier les informations de l'utilisateur.
3. Si les informations sont valides, on appelle l'API de la banque pour procéder au paiement. L'API nous retourne alors un identifiant de paiement permettant de récupérer ses informations, si besoin.
4. On stocke l'identifiant de paiement en base de données afin de pouvoir récupérer les informations du paiement en cas de problème.
5. Si tout s'est bien passé, on affiche un message de succès à l'utilisateur.

Réalisez le diagramme de séquence du scénario où tout se passe bien.

X. Essentiel

XI. Auto-évaluation

A. Exercice final

Exercice 1

[solution n°5 p.34]

Exercice

À quoi sert le diagramme de cas d'utilisation ?

- ○ À montrer l'enchaînement des étapes d'une fonctionnalité
- ○ À lister les fonctionnalités et les liens entre elles
- ○ À définir les interactions entre les différents systèmes

Exercice

Dans un diagramme de cas d'utilisation, quels éléments peuvent être désignés comme acteurs ?

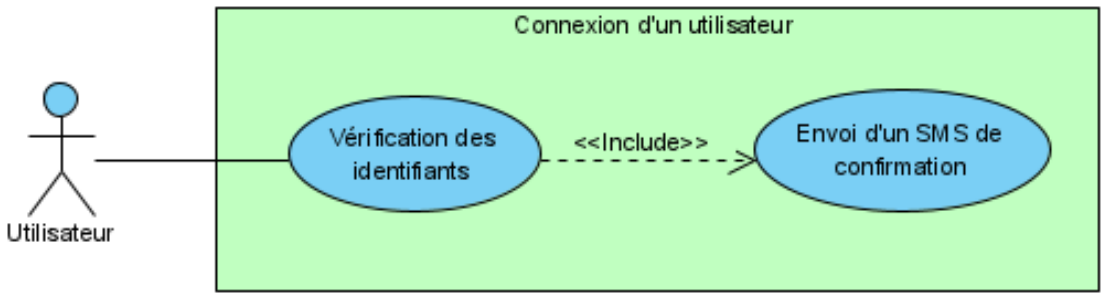
- ☐ Une fonctionnalité
- ☐ Un utilisateur
- ☐ Un système externe
- ☐ Le disque dur du serveur
- ☐ Une tâche planifiée

1 https://www.uml-diagrams.org/sequence-diagrams.html
2 https://www.uml-diagrams.org/timing-diagrams.html

Tous droits réservés à STUDI - Reproduction interdite

Exercice

Laquelle de ces affirmations concernant le diagramme suivant est vraie ?



- Le cas d'utilisation « Envoi d'un SMS » peut parfois être exécuté au moment de la vérification des identifiants
- Le cas d'utilisation « Envoi d'un SMS » est toujours exécuté au moment de la vérification des identifiants
- Le cas d'utilisation « Envoi d'un SMS » est une variante de la vérification des identifiants
- Si la vérification des identifiants échoue, alors on doit envoyer un SMS de confirmation

Exercice

Quel diagramme permet d'avoir une vue d'ensemble du projet en regroupant les cas d'utilisation dans des ensembles ?

- Le diagramme de paquetages
- Le diagramme d'activité
- Le diagramme global d'interactions

Exercice

À quoi sert le diagramme d'activité ?

- À montrer les communications entre plusieurs systèmes, pour voir quand ils sont actifs
- À établir la liste des étapes nécessaires à la réalisation d'un cas d'utilisation
- À définir tous les états possibles d'un composant

Exercice

Lequel de ces éléments n’apparaît pas dans un diagramme d'activité ?

- Le nœud initial
- Le nœud de décision
- La transition
- L'acteur
- L'action

Exercice

Tous droits réservés à STUDI - Reproduction interdite

Combien de diagrammes de séquence faut-il faire pour que les spécifications fonctionnelles soient complètes ?

- Un par projet
- Un par cas d'utilisation
- Un par scénario possible de chaque cas d'utilisation
- Un par étape de chaque cas d'utilisation, telles qu'elles ont été définies dans le diagramme d'activité

Exercice

Quel est l'avantage du diagramme de temps par rapport au diagramme de séquence ?

- Il permet de voir le temps précis nécessaire à une action, ce qui peut permettre d'identifier les points de blocage
- Il permet de rassembler tous les diagrammes de séquence en un seul
- Il permet de voir toutes les communications entre les systèmes

Exercice

Lequel de ces diagrammes est également utilisé dans les spécifications techniques ?

- Le diagramme de cas d'utilisation
- Le diagramme de paquetages
- Le diagramme de séquence

Exercice

À quelle catégorie appartiennent les diagrammes vus dans ce cours ?

- Les diagrammes de structure
- Les diagrammes de comportement

B. Exercice : Défi

Un client souhaiterait créer sa propre plateforme de réservation de places de concert pour sa ville, qui contient deux salles de concert : la salle Georges Brassens et la salle Vivaldi. Actuellement, les deux salles possèdent chacune leur propre système de gestion des réservations. Le but du projet est donc d'avoir une application permettant de centraliser les informations pour que les utilisateurs puissent retrouver toutes les informations au même endroit.

Grâce à vos nouvelles connaissances, vous allez devoir réaliser les spécifications fonctionnelles de ce projet.

Question 1

[solution n°6 p.36]

L'application doit permettre de visualiser les informations de tous les concerts se déroulant dans les deux salles, et de réserver la place de leur choix. Les visiteurs non connectés pourront visualiser les informations, mais seuls les utilisateurs connectés pourront réserver une place. Une fois la réservation de la place faite, un utilisateur peut choisir de se faire envoyer sa place de concert par la Poste.

Chaque jour, à minuit, une tâche automatisée va contacter les API des deux salles de concert pour récupérer les informations des événements à venir (date, prix des billets, nombre de places disponibles...). Ces informations seront stockées dans la base de données de l'application.

Réalisez le diagramme de cas d'utilisation de cette application.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

[solution n°7 p.37]

La partie la plus complexe de cette application est l'appel aux API des salles. Ces deux appels doivent être réalisés en parallèle, mais ont des spécifications différentes :

- L'API de la salle Georges Brassens utilise un système de tokens : l'application doit envoyer un code secret (appelé *token*) pour s'authentifier auprès de l'API. Une fois cette étape réalisée, et si le token est validé par l'API, alors il est possible de récupérer les informations des concerts.
- L'API de la salle Vivaldi est plus ouverte : il est possible de récupérer les informations directement, sans avoir à s'authentifier.

En plus du cas du token invalide, chaque étape de communication avec une des API peut retourner une erreur, dans le cas où l'API est injoignable, par exemple. Dans ce cas, il faut insérer l'erreur dans les logs du serveur.

Une fois les données des deux API récupérées, l'application doit d'abord trier les informations pour éviter d'insérer les données d'un concert qui serait déjà en base. Une fois le tri effectué, il ne reste plus qu'à insérer les informations dans la base de données.

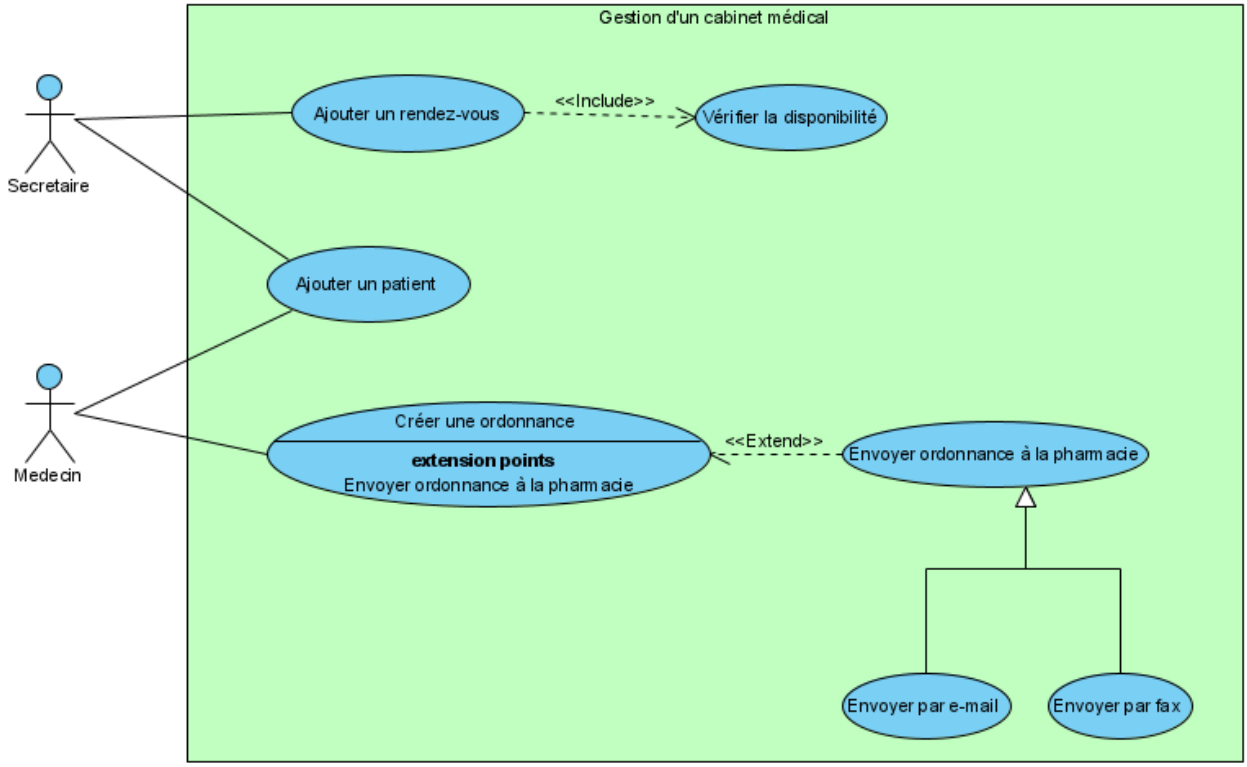
Réalisez le diagramme d'activité de cette fonctionnalité.

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

p. 9 **Solution n°1**

- L'application possède deux acteurs : les secrétaires et les médecins. Il y a trois fonctionnalités de base : la prise de rendez-vous (accessible par les secrétaires), la création d'une ordonnance (accessible par les médecins) et la création de la fiche du patient (accessibles par les deux).
- À chaque prise de rendez-vous, il faut obligatoirement vérifier que le créneau soit disponible. « Vérifier la disponibilité » est donc un cas d'utilisation qui est inclus par le cas « prise de rendez-vous ».
- L'envoi d'une ordonnance, au contraire, n'est pas obligatoire. « Envoyer l'ordonnance » est donc un cas d'utilisation qui étend le cas « création d'une ordonnance ».
- Il existe plusieurs moyens pour envoyer une ordonnance : par e-mail et par fax. Il est possible de représenter ce choix en spécialisant le cas « envoi d'une ordonnance » en deux autres cas d'utilisation : « envoyer par e-mail » et « envoyer par fax ».

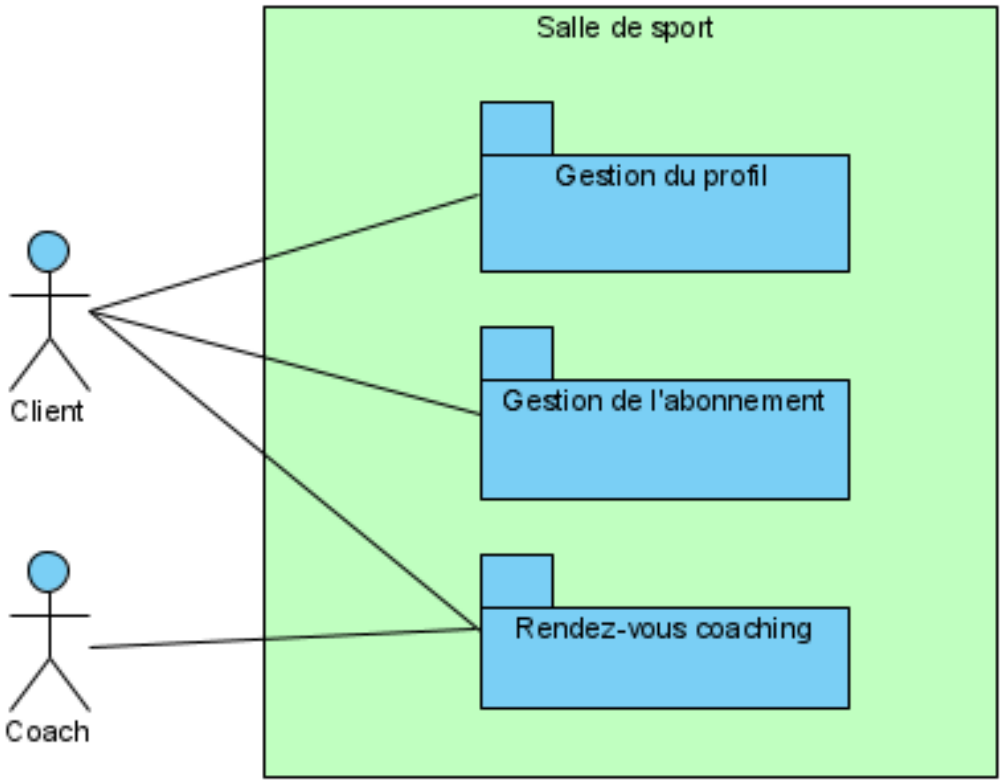


p. 12 **Solution n°2**

Toutes ces fonctionnalités peuvent être regroupées en trois paquetages : la gestion du profil d'un client (création et modification du profil), la gestion de l'abonnement (renouvellement et résiliation) et la gestion des rendez-vous coaching (création, annulation, affichage du planning).

La gestion du profil et celle de l'abonnement sont accessibles uniquement par l'acteur **Client**. Les rendez-vous coaching, eux, sont accessibles par les deux : même si certaines fonctionnalités ne seront accessibles que par l'un ou par l'autre, le paquetage est lié aux deux.

Tous droits réservés à STUDI - Reproduction interdite



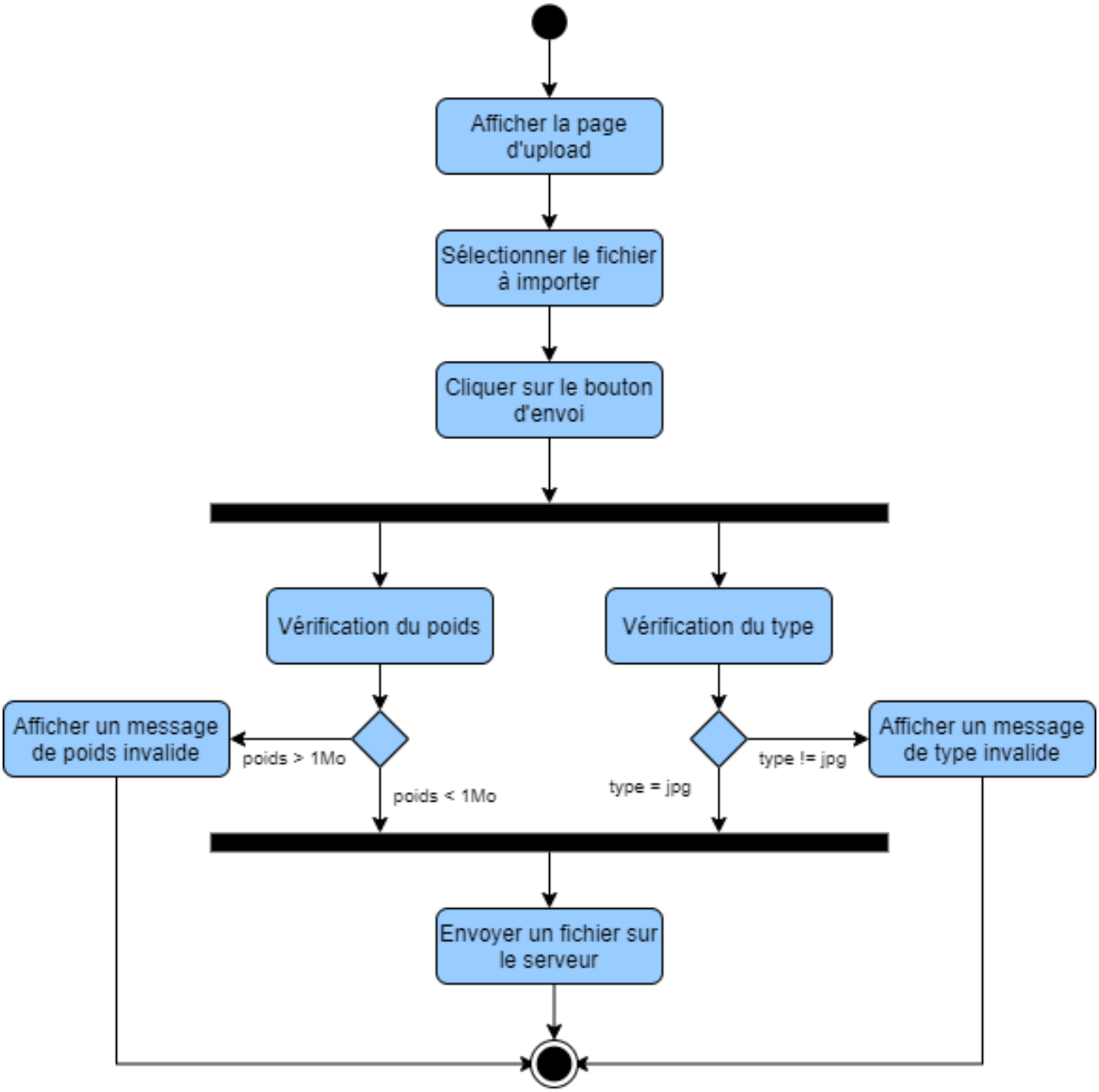
p. 19 **Solution n°3**

Les trois premières actions sont classiques : on affiche la page à l'utilisateur, qui sélectionne un fichier sur son ordinateur et clique sur le bouton d'envoi de l'image (facultatif, on peut imaginer que les actions se font au moment où le fichier a été renseigné).

Deux actions bifurquent ensuite : la vérification du poids et celle du type. Chaque vérification possède son nœud de décision et son message d'erreur en cas de problème.

Enfin, une fois que les deux vérifications sont terminées et qu'on sait que le fichier est conforme, on utilise un nœud de fusion pour finaliser l'envoi de l'image sur le serveur.

Tous droits réservés à STUDI - Reproduction interdite

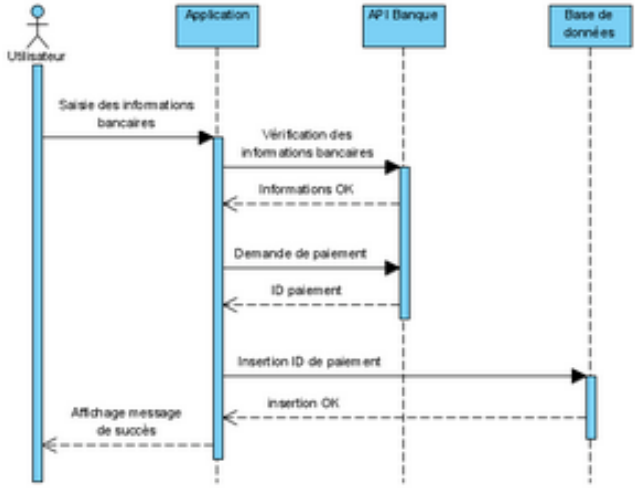


p. 26 **Solution n°4**

Tous droits réservés à STUDI - Reproduction interdite

Il y a 4 acteurs : l'utilisateur, l'application, l'API de la banque et la base de données.

L'application va d'abord devoir appeler deux fois l'API de la banque : une fois pour vérifier les informations, et une fois pour procéder au paiement. Une fois l'ID de paiement reçu, l'application va ensuite l'insérer dans la base de données. Ces étapes sont décrites par le diagramme suivant :



Exercice p. 26 **Solution n°5**

Exercice

À quoi sert le diagramme de cas d'utilisation ?

- ○ À montrer l'enchaînement des étapes d'une fonctionnalité
- ◉ À lister les fonctionnalités et les liens entre elles
- ○ À définir les interactions entre les différents systèmes

Le diagramme de cas d'utilisation sert à lister de la manière la plus exhaustive possible les fonctionnalités de l'application.

Exercice

Dans un diagramme de cas d'utilisation, quels éléments peuvent être désignés comme acteurs ?

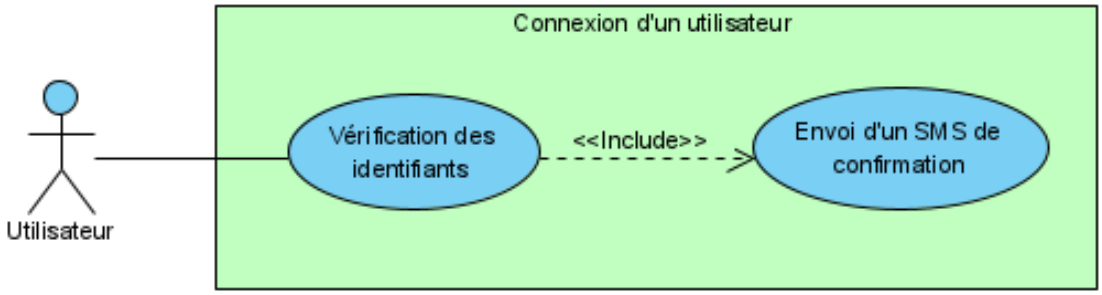
- ☐ Une fonctionnalité
- ☑ Un utilisateur
- ☑ Un système externe
- ☐ Le disque dur du serveur
- ☑ Une tâche planifiée

Le terme « acteur » désigne tout ce qui peut interagir avec notre application. La plupart du temps, ce terme désigne un utilisateur, mais il peut également désigner un système externe si notre application dispose d'une API, ou une tâche planifiée.

Exercice

Laquelle de ces affirmations concernant le diagramme suivant est vraie ?

Tous droits réservés à STUDI - Reproduction interdite



- ○ Le cas d'utilisation « Envoi d'un SMS » peut parfois être exécuté au moment de la vérification des identifiants
- ◉ Le cas d'utilisation « Envoi d'un SMS » est toujours exécuté au moment de la vérification des identifiants
- ○ Le cas d'utilisation « Envoi d'un SMS » est une variante de la vérification des identifiants
- ○ Si la vérification des identifiants échoue, alors on doit envoyer un SMS de confirmation

Une relation d'inclusion entre deux cas d'utilisation permet d'indiquer que le cas inclus est obligatoirement réalisé avec le cas de base.

Exercice

Quel diagramme permet d'avoir une vue d'ensemble du projet en regroupant les cas d'utilisation dans des ensembles ?

- ◉ Le diagramme de paquetages
- ○ Le diagramme d'activité
- ○ Le diagramme global d'interactions

Le diagramme de paquetages permet de regrouper les cas d'utilisation dans des paquetages (aussi appelés *packages*).

Exercice

À quoi sert le diagramme d'activité ?

- ○ À montrer les communications entre plusieurs systèmes, pour voir quand ils sont actifs
- ◉ À établir la liste des étapes nécessaires à la réalisation d'un cas d'utilisation
- ○ À définir tous les états possibles d'un composant

Le diagramme d'activité permet de connaître toutes les étapes qui composent un cas d'utilisation.

Exercice

Lequel de ces éléments n'apparaît pas dans un diagramme d'activité ?

- ○ Le nœud initial
- ○ Le nœud de décision
- ○ La transition
- ◉ L'acteur

Tous droits réservés à STUDI - Reproduction interdite

- ○ L'action

Q Le diagramme d'activité ne s'intéresse pas à l'acteur qui réalise la fonctionnalité, seulement à ses étapes.

Exercice

Combien de diagrammes de séquence faut-il faire pour que les spécifications fonctionnelles soient complètes ?

- ○ Un par projet
- ○ Un par cas d'utilisation
- ◉ Un par scénario possible de chaque cas d'utilisation
- ○ Un par étape de chaque cas d'utilisation, telles qu'elles ont été définies dans le diagramme d'activité

Q Un diagramme de séquence permet de montrer les interactions entre plusieurs systèmes pour un scénario précis d'un cas d'utilisation. Pour que les spécifications soient complètes, il faut donc un diagramme par scénario. Attention, il ne faut pas confondre le diagramme de séquence avec le diagramme global d'interaction, qui est un mélange entre le diagramme de séquence et le diagramme d'activité, et qui décompose les interactions de chaque étape.

Exercice

Quel est l'avantage du diagramme de temps par rapport au diagramme de séquence ?

- ◉ Il permet de voir le temps précis nécessaire à une action, ce qui peut permettre d'identifier les points de blocage
- ○ Il permet de rassembler tous les diagrammes de séquence en un seul
- ○ Il permet de voir toutes les communications entre les systèmes

Q Le diagramme de temps utilise un axe temporel précis, ce qui lui permet de savoir quelles étapes prennent le plus de temps dans un système.

Exercice

Lequel de ces diagrammes est également utilisé dans les spécifications techniques ?

- ○ Le diagramme de cas d'utilisation
- ◉ Le diagramme de paquetages
- ○ Le diagramme de séquence

Q En plus de regrouper les cas d'utilisation, le diagramme de paquetages peut également permettre de regrouper les classes en *packages*, ce qui permet de structurer le code.

Exercice

À quelle catégorie appartiennent les diagrammes vus dans ce cours ?

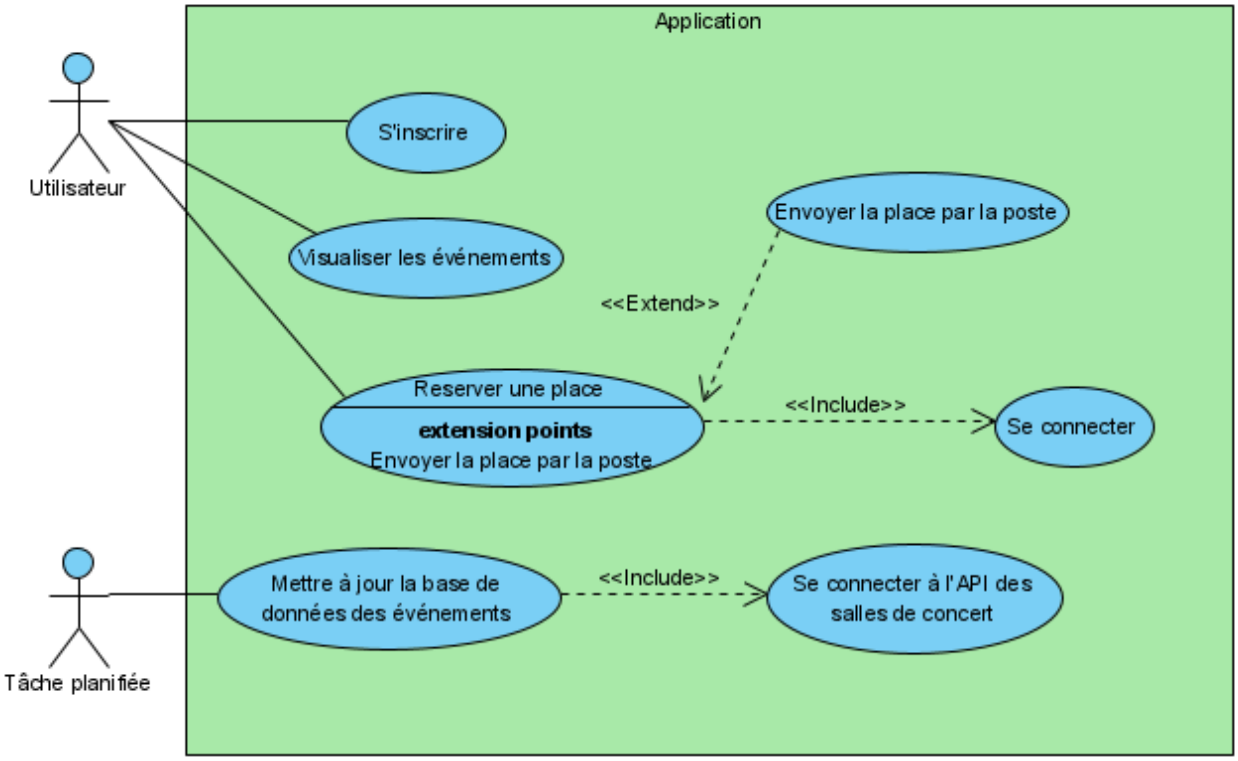
- ○ Les diagrammes de structure
- ◉ Les diagrammes de comportement

Q Les diagrammes que nous venons de voir sont des diagrammes de comportement. Ils sont utilisés dans les spécifications fonctionnelles.

p. 28 **Solution n°6**

Tous droits réservés à STUDI - Reproduction interdite

L'application possède deux types d'acteurs : les utilisateurs et la tâche planifiée. La réservation d'une place de concert nécessite de se connecter, c'est donc une inclusion ; tandis que l'envoi d'une place par la Poste est une extension, puisqu'elle n'est pas obligatoire.



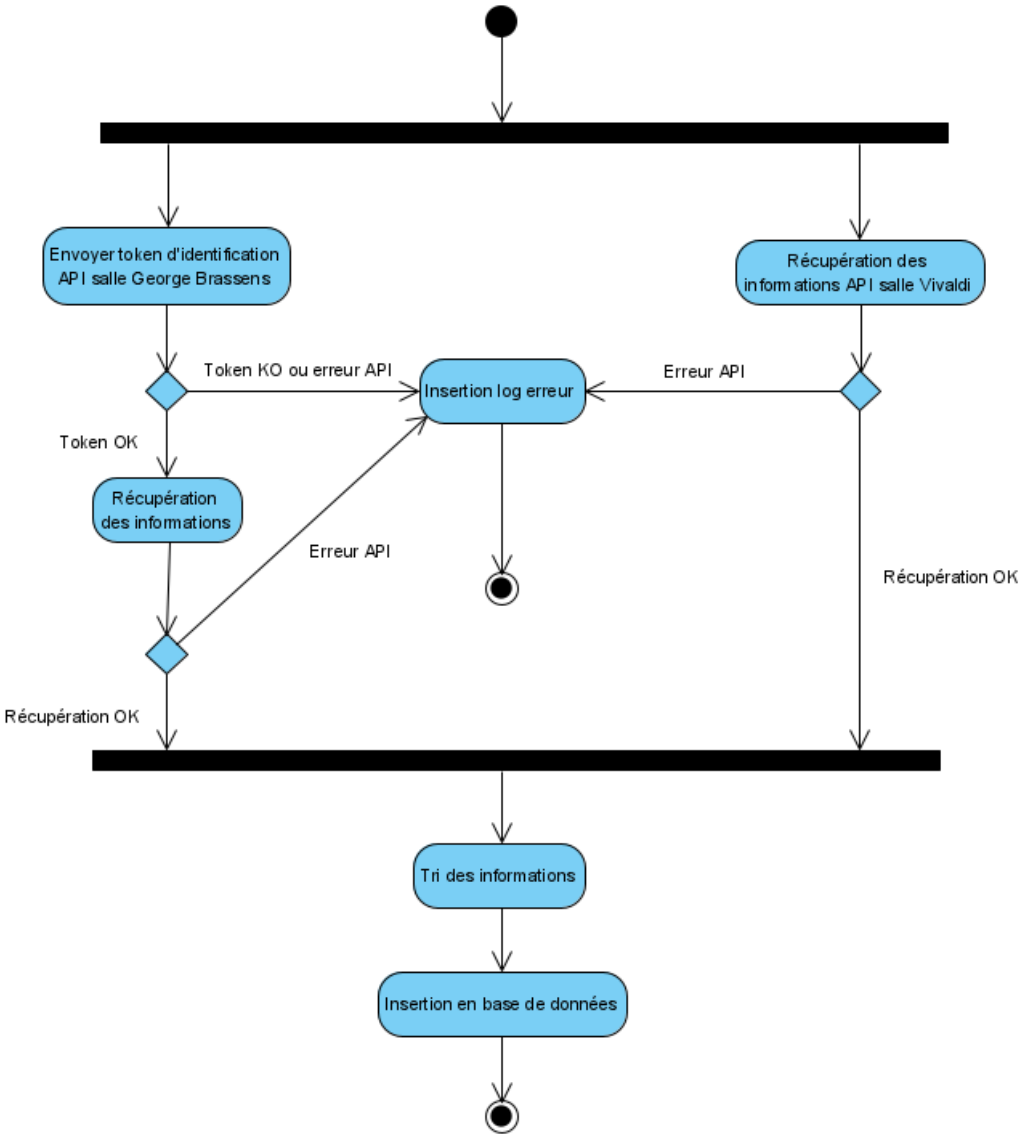
Il aurait également été possible de distinguer les utilisateurs connectés et les visiteurs en deux acteurs différents : une telle proposition ne rendrait pas le schéma faux.

p. 29 Solution n°7

Le diagramme d'activité commence par un nœud de bifurcation, représentant les appels parallèles aux deux API. Chaque étape de communication avec les API doit avoir son propre nœud de décision afin de gérer une erreur de communication.

La récupération des informations des deux API se rejoignent en un nœud de fusion, avant de faire le tri et l'insertion en base de données.

Tous droits réservés à STUDI - Reproduction interdite



Tous droits réservés à STUDI - Reproduction interdite